# Ansible Automation Platform 2.7

## *Automation Portal and Content Lifecycle*

Version 1.0.0, 2026-10-13

# Home

## Introduction

The Ansible Automation Portal with Ansible Automation Platform (AAP) serves as a unified interface that shifts the platform focus from simple execution to a comprehensive "build, discover, and run" experience. It simplifies enterprise automation for users of all skill levels by replacing complex CLI workflows with a streamlined, point-and-click web experience.

Duration: 1 hours

## Topics covered in this course

- Automation Portal
- Content Lifecycle
- Execution Environment (EE) Builder

## Prerequisites

This course assumes that you have the following prior experience:

- Basic understanding of Ansible Automation Platform and OpenShift Container Platform (OCP).

## Contributors

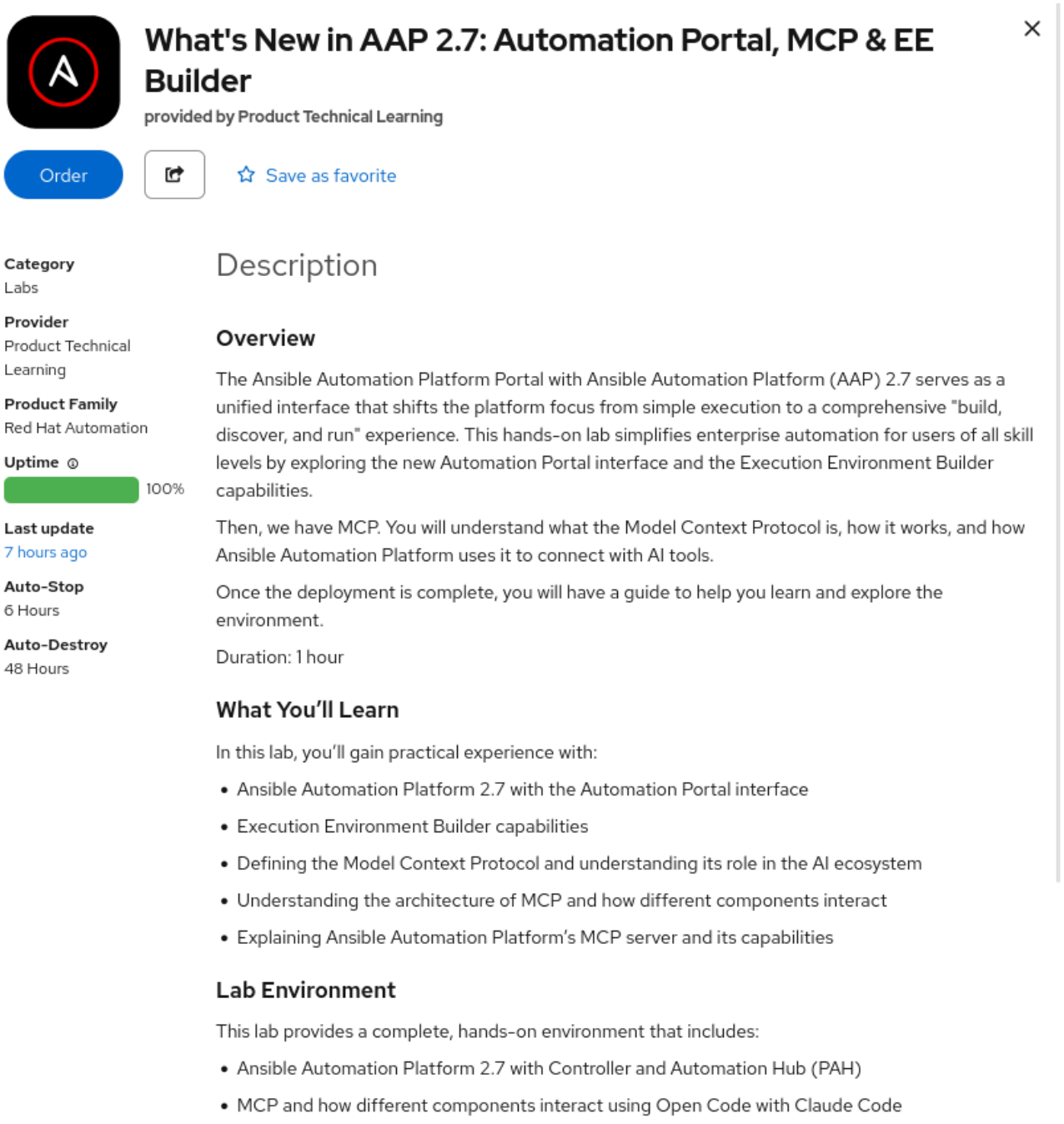
The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- Amrinder Singh (Content Architect)
- Anna Swicegood (Learning Product Manager)

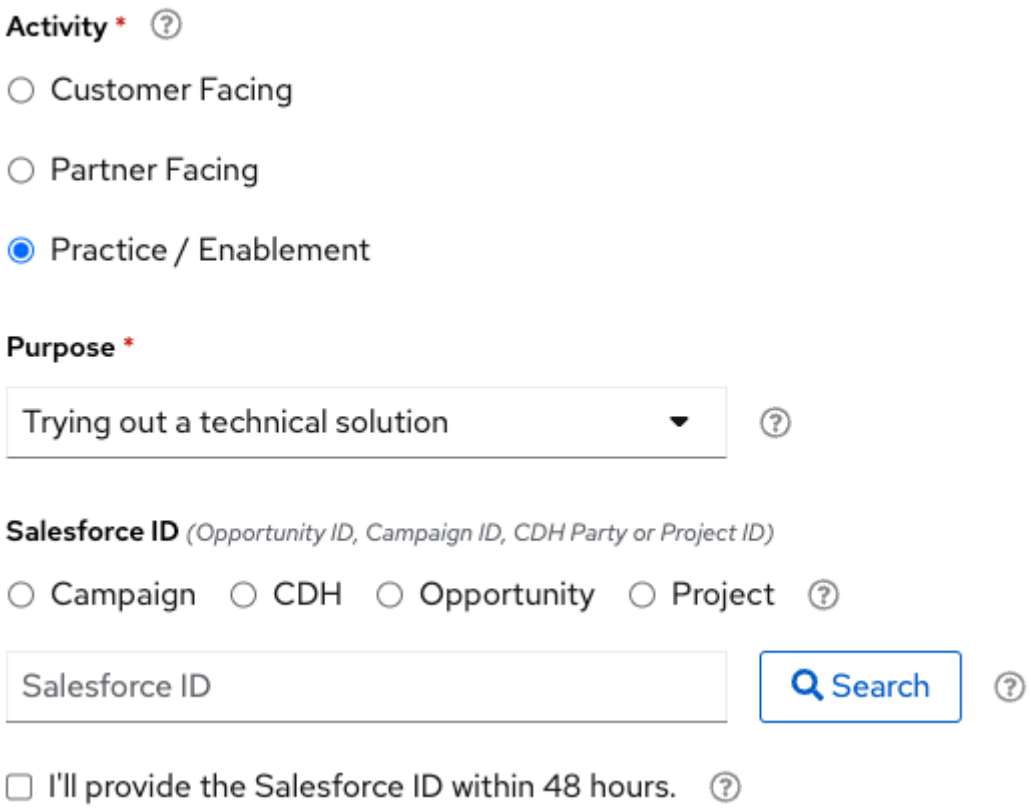
# Lab Environment

## Instructions to Launch Your Lab on the Red Hat Demo Platform (RHDP)

1. **Log in** to the RHDP portal.
2. **Search** for the catalog entry: **What's New in AAP 2.7: Automation Portal, MCP & EE Builder**.




3. **Click** on the catalog entry in the search results.
4. On the catalog page, **click** the **Order** button.
5. **Fill out** the required details in the order form.




6. **Review the warning** at the bottom of the form and **check the box** labeled: **"I confirm that I understand the above warnings."**
7. **Click** the **Order** button to place your lab order.

## Important Notes:

- This lab may take approximately **20 minutes** to become ready.
- You will receive an **email with access details** once your lab environment is ready.
- You can also **retrieve lab access** directly from the RHDP portal.

## How to Access Your Lab via the RHDP Portal:

1. On the RHDP portal, **click** on the **Services** option in the left-hand menu.
2. **Select your lab** from the listings on the right-hand side of the page to view access details.

## RHDP Portal Links

- RedHat associates: https://demo.redhat.com/
- RedHat partners: https://partner.demo.redhat.com/

# Automation Portal

The Ansible Automation Portal with Ansible Automation Platform (AAP) serves as a unified interface that shifts the platform focus from simple execution to a comprehensive "build, discover, and run" experience. It simplifies enterprise automation for users of all skill levels by replacing complex CLI workflows with a streamlined, point-and-click web experience.

Key features include:

- **Simplified User Experience:** Designed for business users, reducing the need for deep technical knowledge of Ansible playbooks.
- **Platform Integration:** Deployed on OpenShift Container Platform (OCP) with support for RBAC-based navigation, allowing administrators to reduce UI complexity through role-based access policies.
- **Role-Based Access Control:** Facilitates user role separation, ensuring that users only interact with relevant automation assets.
- **Unified Workflow:** Acts as a centralized hub for managing the full lifecycle of automation, from building Execution Environments to discovering and running pre-defined automation content.

# Automation Portal: Installation Overview with Architectural Benefits

The Automation Portal with Ansible Automation Platform provides a centralized interface for managing automation workflows, visual environment building, and content discovery. When deploying on OpenShift Container Platform (OCP), the Automation Portal leverages the Kubernetes-native architecture to ensure scalability, high availability, and seamless integration with existing AAP controller components.

## The Evolution of the Portal

In previous iterations, such as AAP 2.6/7, the portal functioned primarily as an execution layer. Users navigated to the interface, identified authorized job templates, populated surveys, and triggered automation.

Automation Portal 2.7 expands this scope to support three primary pillars:

- **Visual Execution Environment (EE) Builder:** A GUI-driven workflow for container image creation.
- **Centralized Content Discovery:** A unified catalog that aggregates automation assets from multiple sources.
- **Platform-wide Governance:** Enhanced RBAC controls, administrative navigation policies, and AI-assisted operational workflows.

## Architectural Benefits

The shift toward this "Build, Discover, and Run" architecture provides several key advantages for organizations:

#### 1. Unified Developer Productivity

By integrating the EE Builder and the Content Discovery catalog directly into the portal, developers no longer need to switch context between the command line, CI/CD pipelines, and the execution interface. This reduces the "friction of automation," allowing teams to define, build, and deploy automation assets without leaving the AAP ecosystem.

#### 2. Reduced Complexity via Admin-Defined RBAC

The architectural design of the 2.7 portal emphasizes role-based navigation. Administrators can configure custom menus to match user roles: * **End-Users:** May see a simplified view restricted to "Templates" and "Job History," minimizing cognitive load and accidental configuration changes. * **Platform Engineers:** Retain access to the full portal, including builder tools, catalog management, and administrative policy settings.

#### 3. Streamlined Governance and Discovery

The architecture supports multi-source aggregation. By centralizing content from various repositories (GitHub, GitLab, Private Automation Hub), the portal provides a "browse-before-build" workflow. This allows teams to trace existing automation assets, preventing the duplication of effort and ensuring that teams are using approved, organization-wide standards.

#### 4. Extensibility and AI Readiness

With the inclusion of an MCP (Model Context Protocol) server, the portal is architected to support AI-assisted operations. This allows for natural language interaction with the platform, enabling users to query automation status or trigger workflows through supported AI clients, further lowering the barrier to entry for non-technical stakeholders.

By consolidating these functions, Automation portal reduces the operational overhead associated with maintaining disparate build tools and content repositories, ensuring that automation scaling remains consistent with enterprise security and governance policies.

## Prerequisites and Requirements

Before initiating the deployment, ensure your environment meets the following technical requirements:

- **Platform Subscription:** Valid Red Hat Ansible Automation Platform subscription.
- **Existing AAP Infrastructure:** An active instance of AAP 2.6+ to facilitate authentication and integration.
- **OpenShift Environment:** Access to an OpenShift Container Platform cluster (version 4.12–4.17+).
- **Permissions:** Cluster-level permissions to create OAuth applications within the target project.

- **CLI Tools:**
- `oc` CLI tool for OpenShift management.
- `helm` (version 3.10 or newer) for package deployment.

### Installation Overview

The official deployment method uses **Helm charts** on OpenShift Container Platform. The Helm chart automates the deployment of the Automation Portal, handling networking, storage, and integration with your AAP instance.and the Operator based Deployment method is also available for the Automation Portal. The Operator simplifies the installation and management of the portal by providing a declarative approach to deployment, allowing for easier upgrades and configuration management.



We will be using the Helm chart method for the Automation Portal deployment in this lab.



**Two-Phase Installation Process:**

1. **Phase 1 - Prerequisites:** Complete set up OAuth, API tokens,OC cli and OpenShift secrets configuration.
2. **Phase 2 - Deployment:** Deploy the Automation Portal via Helm chart, configure the necessary environment variables, and validate the installation.

Both phases are required for a successful installation.

# Managing RBAC-based Navigation Menus for User Role Separation

In modern enterprise automation, `platform sprawl` is a significant hurdle. When users are presented with a cluttered interface containing tools they don't need—such as Execution Environment (EE) builders or Git repository management—it increases cognitive load and introduces potential security risks.

The Automation Portal addresses this by providing **RBAC-based navigation menus**, allowing platform architects to tailor the UI experience based on the user's specific role.



**Related Pages:** This page focuses on controlling which menu items are visible.

### Architectural Concept

Ansible automation portal and Ansible Automation Platform use separate but related permission systems. Understand the permission model

**Ansible automation portal RBAC:**

- Controls which users can view templates in the Ansible automation portal catalog.

- Controls which users can access portal templates and submit jobs.
- Controls which navigation items are visible in the sidebar.

**Ansible Automation Platform RBAC:**

- **Controls synchronization scope:** Only Ansible Automation Platform job templates accessible by the configured API token (`ansible.rhaap.token`) are synchronized to Ansible automation portal.
- **Controls auto-generated template visibility:** Ansible Automation Platform permissions determine whether authenticated users can view and execute auto-generated templates in Ansible automation portal. Custom templates are not filtered by Ansible Automation Platform permissions.
- **Validates execution permissions:** When a user executes any template, Ansible Automation Platform checks that user's execute permissions on the underlying job template before launching the job. This applies to both auto-generated and custom templates.

If a user can see a template in the catalog but lacks Ansible Automation Platform execute permissions for the associated job template, the user cannot run the job.

# Reducing UI Complexity through Admin-Defined Access Policies

In large-scale enterprise environments, the Automation Portal can become cluttered with administrative interfaces, repository configurations, and low-level build tools. To improve user experience and maintain security compliance, the Automation Portal allows administrators to define access policies that dynamically tailor the UI based on user roles.



**Related Pages:** This page focuses on API-level permissions and access control. For information on controlling which menu items are visible to users is based on how the RBAC-based setup is done.

## The Concept of UI Minimalism through RBAC

By leveraging Role-Based Access Control (RBAC), you can decouple the complexity of the platform from the needs of the consumer. Reducing UI complexity serves two primary goals:

- **Operational Efficiency:** End users are not overwhelmed by administrative overhead, reducing the cognitive load required to execute automation.
- **Administrative Governance:** By hiding configuration menus (like EE Definitions or Git Repositories), you prevent unauthorized changes to the underlying platform infrastructure.

## Architectural Implementation

The portal maps navigation visibility directly to the user's existing Ansible Automation Platform (AAP) permissions. When a user authenticates, the portal checks their associated permissions:

1. **Identity Mapping:** The portal retrieves the user's roles via their API token.
2. **Policy Evaluation:** The system evaluates the user's role against the defined navigation access policies.
3. **UI Projection:** The frontend dynamically renders only the menu items for which the user has authorized access.



The UI policy does not replace server-side security. The system architecture ensures that even if a UI element were exposed, the AI agent and backend services will block any action not permitted by the server-side configuration.

## Scenario: Role-Based Navigation Configuration

To implement the requested separation of duties, follow this logical configuration structure:

| User Group | Visible Menu Items | Hidden/Restricted Items |
|---|---|---|
| **End Users** | Templates, History | EE Definitions, Collections, Git Repositories, Sync |
| **Platform Engineers** | Full Portal (All Items) | None |

### Configuring the Navigation Policy

To restrict navigation, administrators define access policies within the Portal settings:

1. **Define the Scope:** Identify the target Role (e.g., `cloud-team`) in the AAP RBAC system.
2. **Access Filtering:** Within the Automation Portal administrative settings, navigate to **UI Access Policies**.
3. **Map Groups to Menus:**
   - Create a policy for the `cloud-team`.
   - Set the `Allowed Navigation Items` list to strictly include: `['Templates', 'History']`.
   - Verify that `System Configuration` and `Builder` menus are excluded.

### how to do Step-by-Step Verification

1. **Login as Platform Engineer:** Log in with the `user-admin` account. Verify that the sidebar contains the full suite of options, including **EE Definitions** and **Git Repositories**.
2. **Apply Administrative Filtering:** Navigate to the Portal Admin panel. Select **UI Access Policies** and create a rule for the group containing `user-standard`. Select only **Templates** and **History**.
3. **Login as End User:** Log in with the `user-standard` account. Observe that the navigation sidebar has shrunk, removing all administrative configuration tools.
4. **Verify Backend Protection (Optional):** Attempt to access the URL for the EE Builder directly (e.g., `/portal/ee-builder`). The system should return a 403 Forbidden error, confirming that the UI policy is backed by robust server-side security.

**Troubleshooting**

- **Cache Issues:** If menu changes do not reflect, clear your browser cache or force-refresh the session.
- **Role Propagation:** If a user still sees extra menus, verify that their user role in AAP was updated and their current session token has been refreshed.

# Hands-on Setup: Deploying Automation Portal - Prerequisites and Configuration

In this lab, you'll walk through the essential prerequisite steps to deploy the Ansible Automation Portal on OpenShift Container Platform. We'll configure OAuth authentication with AAP, generate required API tokens, and prepare your OpenShift environment for the portal deployment and in addition to that github configuration to do the setup for the Execution Environments builder Ci/CD configuration to directly push the changes to the Private automation hub. These steps ensure secure communication between the automation portal and your AAP instance.

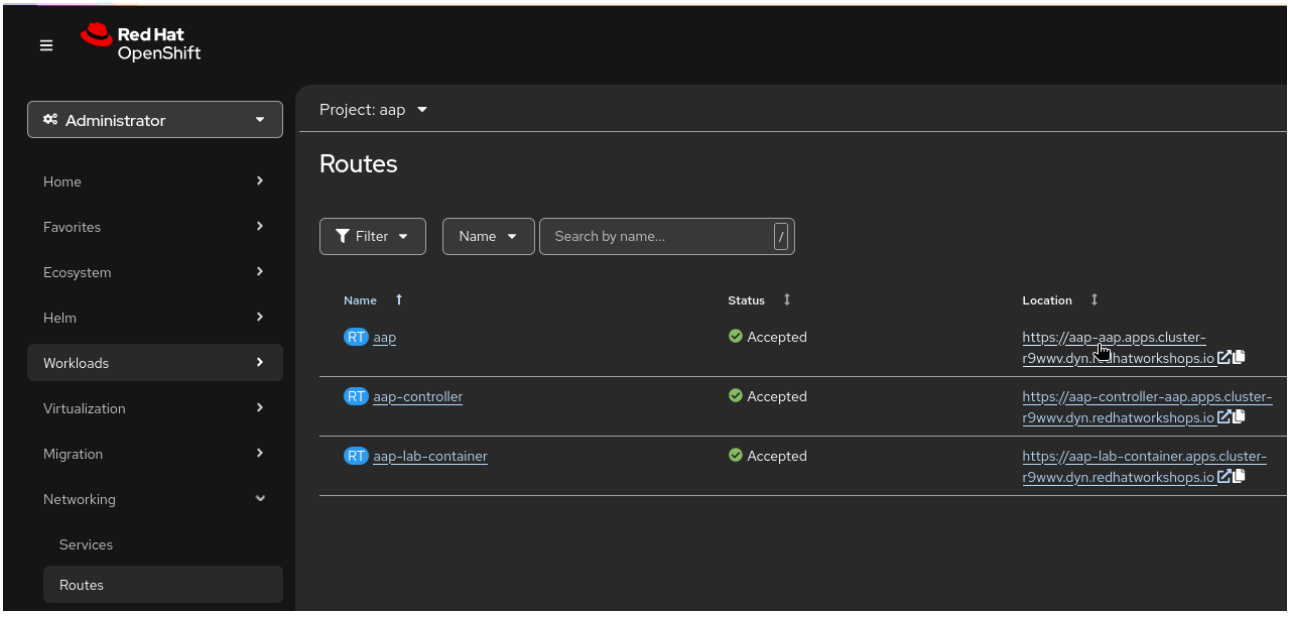
This is the foundational work that must be completed **before** you deploy the Helm chart. Think of it as setting up the "communication channels" between the portal and your AAP instance.

## Task 1: Create an OAuth Application in Ansible Automation Platform

The OAuth application is how the Automation Portal will authenticate users against your AAP instance. Think of it as registering the portal as a "trusted application" within AAP.

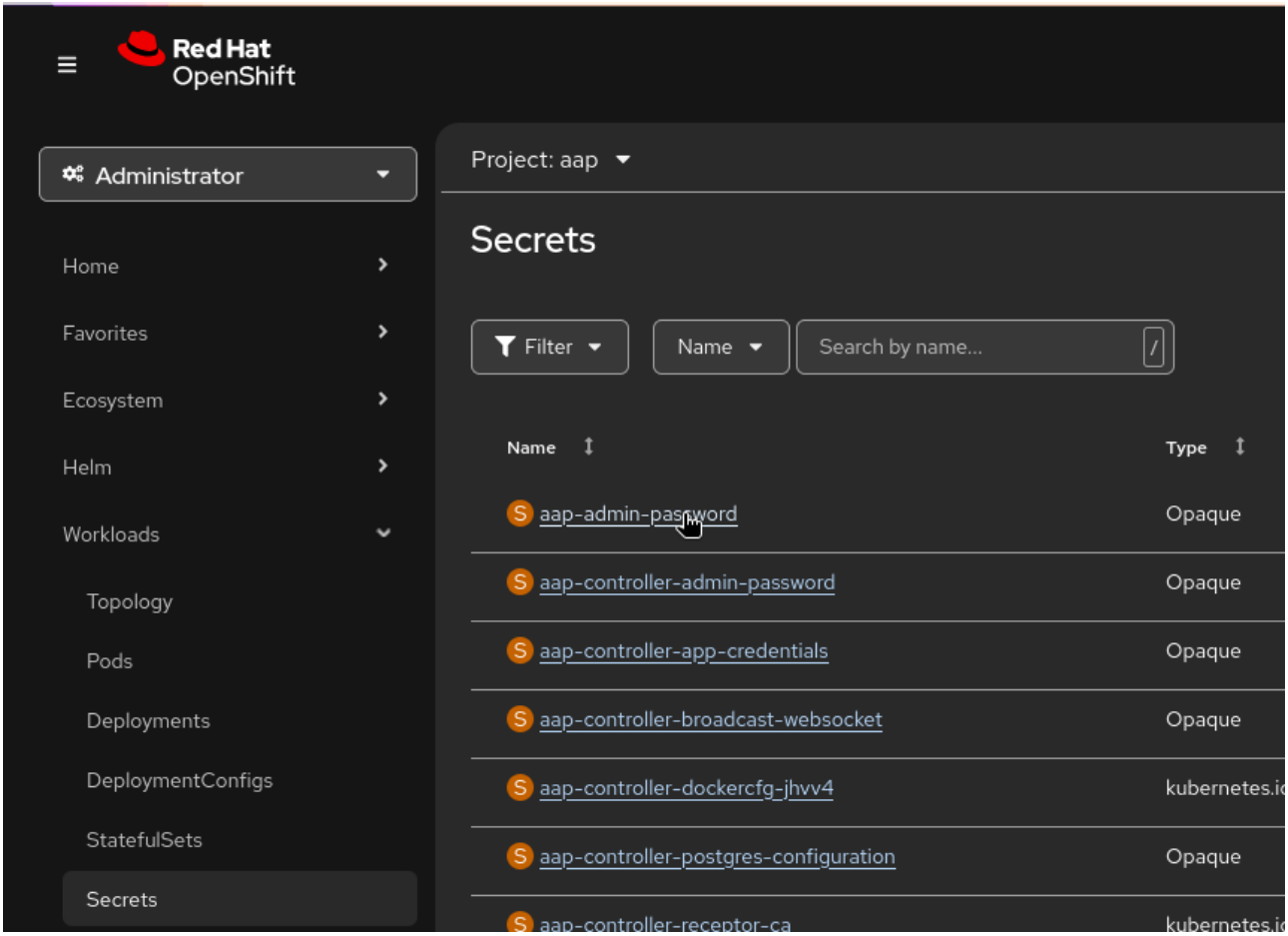
### Step 1.1: Log in to AAP

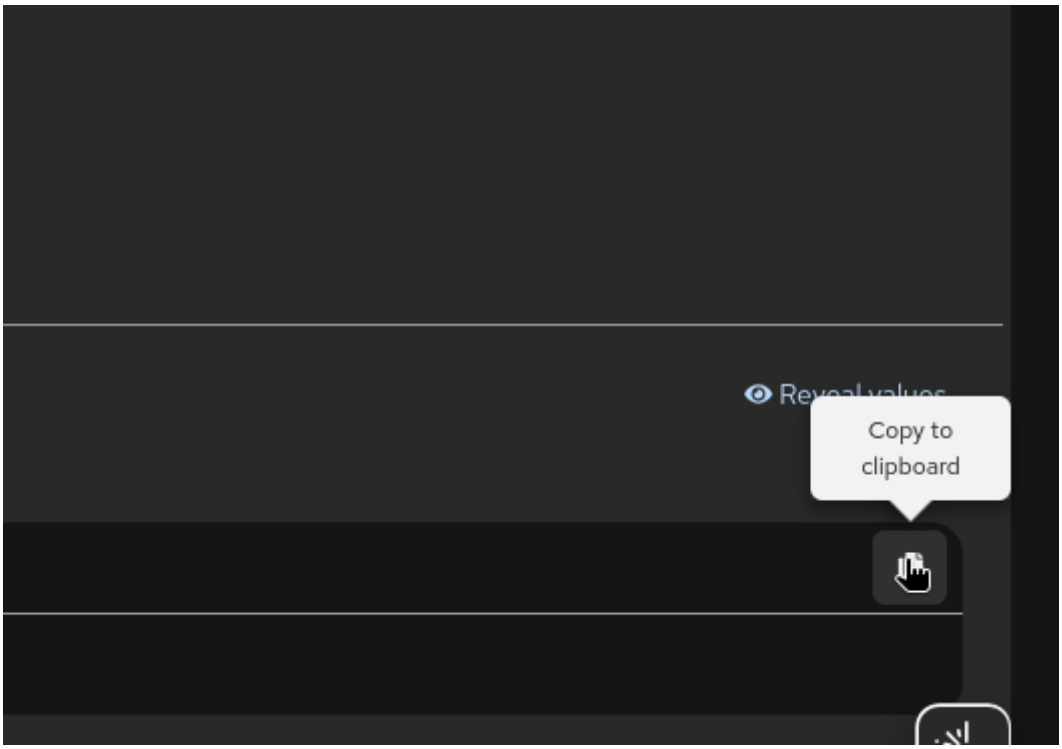
1. From the RHDP login page access the Ansible Automation Platform.
2. Select the project as **aap** and open your Ansible Automation Platform instance in a web browser.




3. Go to `Workload → Secrets → aap-admin-password` and use the `administrator` credentials provided

to you.




Open your Ansible Automation Platform instance in a web browser using RHDP login page and log in with the administrator credentials provided to you.

### Step 1.2: Navigate to OAuth Applications

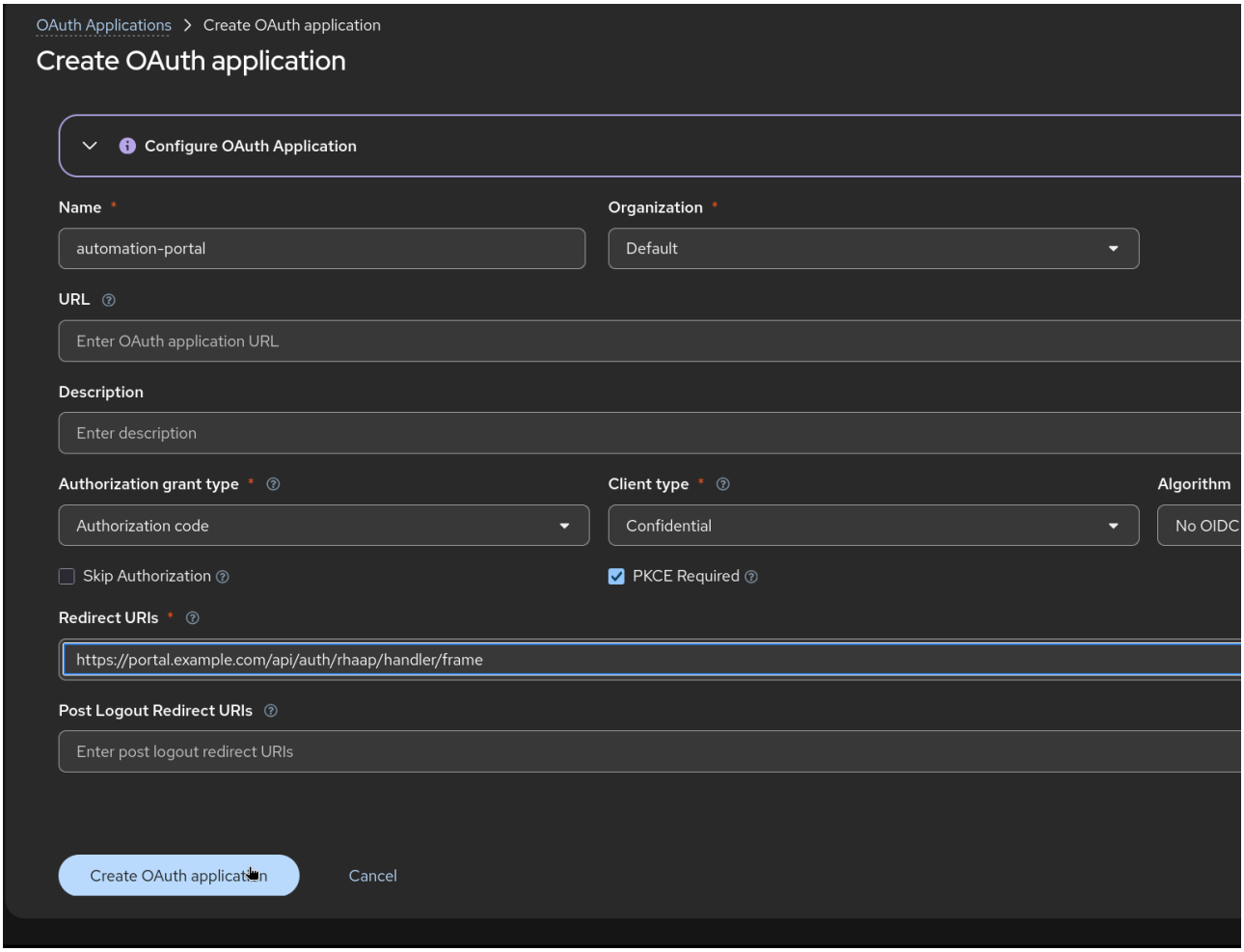
Once logged in:

1. In the left sidebar, click **Access Management**
2. Select **OAuth Applications** from the submenu
3. Click the **Create OAuth Application** button in the upper right corner

### Step 1.3: Configure the OAuth Application

Fill in the following fields with these exact values:

- **Name:** `automation-portal`
- **Organization:** Select the organization where your automation content lives (typically "Default" unless you have a multi-tenant setup)
- **Authorization grant type:** Select **Authorization code**
- **Client type:** Select **Confidential**
- **Redirect URIs:** Enter a placeholder URL for now: `https://portal.example.com/api/auth/rhaap/handler/frame` (we'll update this later with the actual portal deployment URL)






The Redirect URI is temporarily set to a placeholder because the actual portal URL won't exist until after you deploy the Helm chart. We'll come back and update this

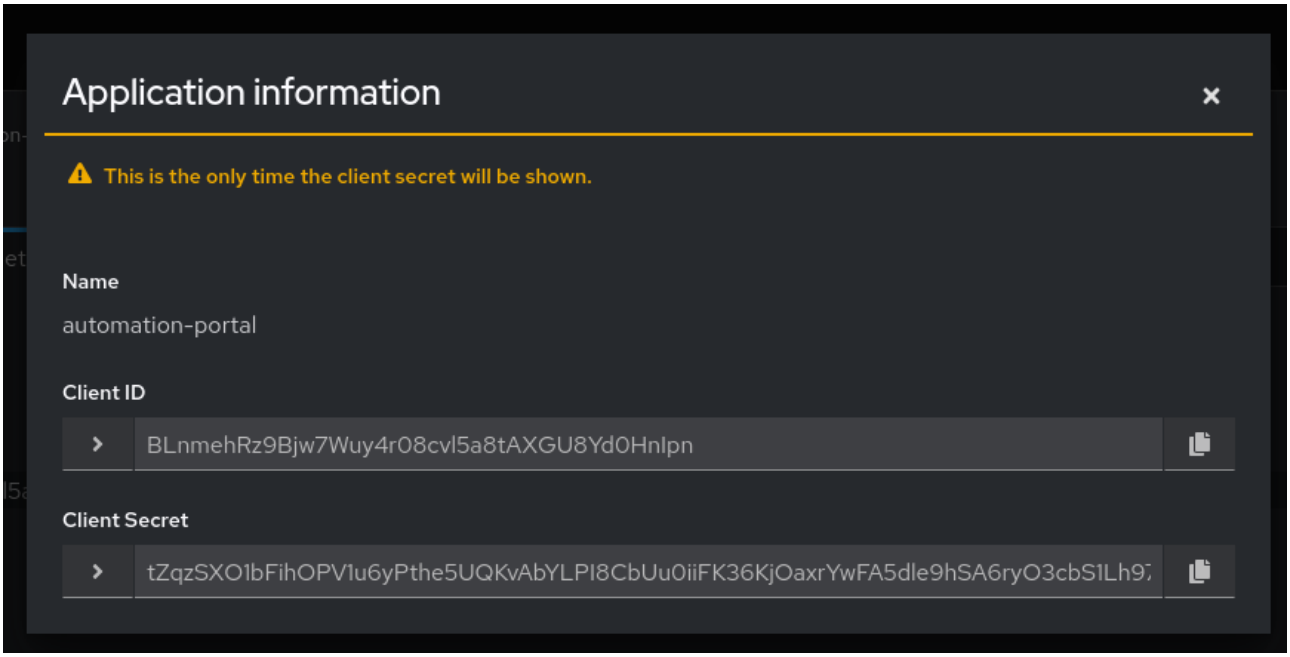
> with the real deployment URL in a later step.

**Step 1.4: Save and Capture Credentials**

Click the **Create OAuth Application** button.

Once created, the system will display:

- **Client ID** — copy this value and save it in a secure location (notepad, password manager, etc.)
- **Client Secret** — this appears only once; copy and save it immediately




> **IMPORTANT:** The Client Secret will never be displayed again. If you lose it, you'll need to regenerate it. Save both values now before navigating away.

## Task 2: Enable OAuth Token Creation for External Users

The Below steps will be only needed if you are using an external identity provider (LDAP, SAML, OAuth, etc.) for user authentication. If your users are logging in directly with AAP credentials, you can skip this step for this lab.

1. In the AAP left sidebar, click **Settings**
2. Select **Platform gateway** from the submenu
3. Look for the setting: **Allow external users to create OAuth2 tokens**

If the setting is not already enabled:

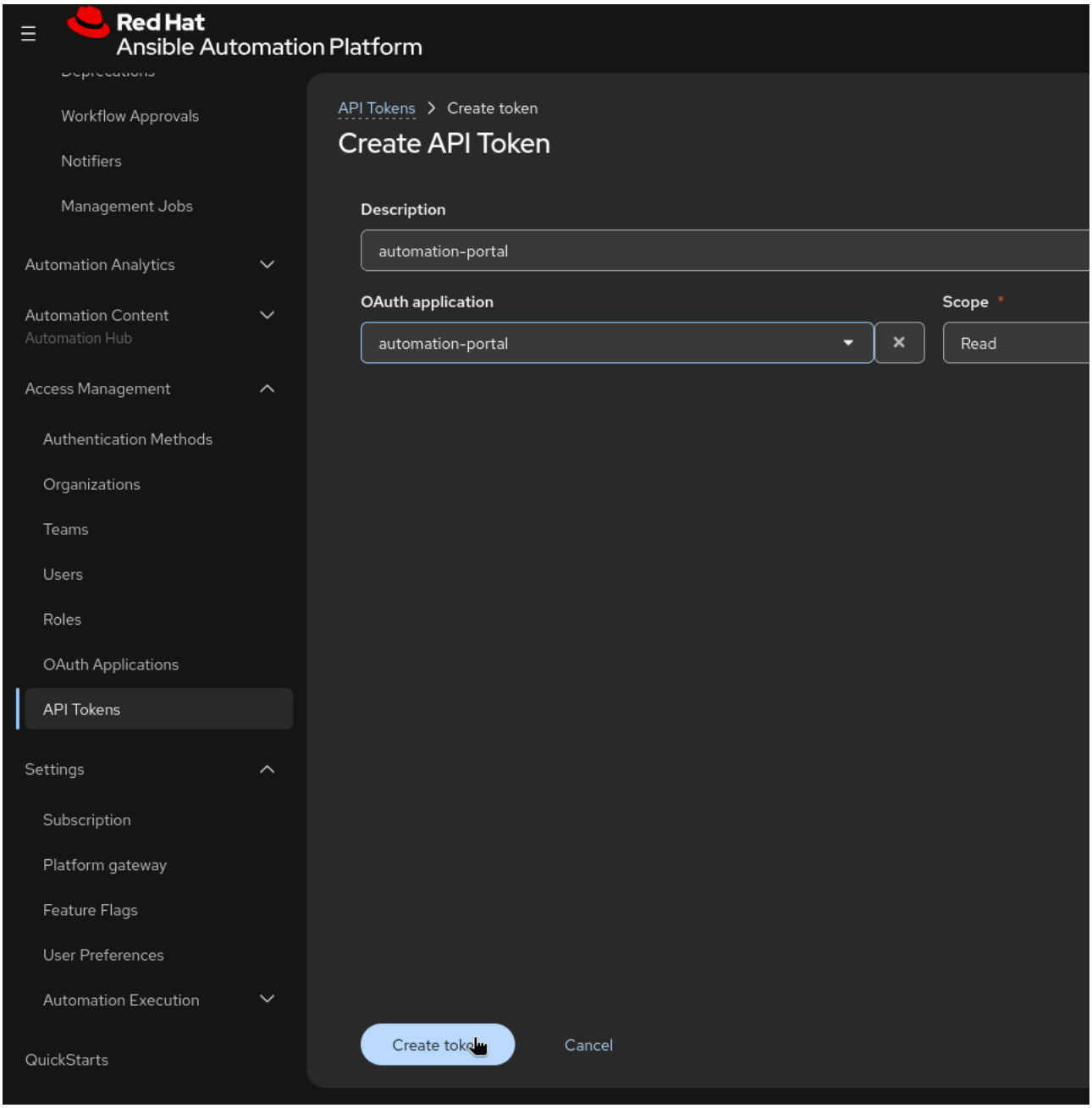
1. Click **Edit platform gateway settings**
2. Toggle **Allow external users to create OAuth2 tokens** to **Enabled**
3. Click **Save platform gateway settings**

Verify that the setting now shows as **Enabled** on the Platform gateway settings page.

## Task 3: Generate an API Token for the Automation Portal

The API token is used by the Automation Portal to communicate with your AAP instance on a service-to-service basis. This token is what allows the portal to synchronize job templates, manage execution environments, and perform automation tasks.

1. Go to the Access Management → API Tokens and




Fill in the following fields:

- **Description:** `automation-portal` (descriptive name for your reference)
- **Application:** Select the **automation-portal** OAuth application you created in Task 1
- **Scope:** Select **read** (the portal needs read access to manage templates and execution environments)

  [5.7] | *5.7.png*

Generate and Save the Token and as the system will display the generated token — copy it immediately and save it in a secure location alongside your Client ID and Client Secret from Task 1.



**IMPORTANT:** Like the Client Secret, the API token is displayed only once. If you navigate away, you cannot retrieve it again without creating a new token.

Make a note of the three values you now have:

- Client ID: _
- Client Secret: _
- API Token: _

These three values are critical for the next phase of setup.

## Task 4: Authenticate with Your OpenShift Cluster

Now we shift focus to the OpenShift side of the setup. We need to connect to your OCP cluster using the CLI tool, which will be the gateway for creating the portal's project and secrets.

1. Login to the bastion host using the credentials from the RHDP login page.
2. Once login to bastion host then login to OCP cli using the command shared in **OpenShift CLI Commands** section on the RHDP login page:

```
[lab-user@bastion ~]$ oc login -u admin -p QpKQDdbdvd91
 https://api.cluster-r9wwv.dyn.redhatworkshops.io:6443
Login successful.

You have access to 83 projects, the list has been suppr
essed. You can list all projects with 'oc projects'

Using project "default".
```

3. Run the following command, replacing `portal` with your preferred project name if you want a different project name(keep it lowercase with hyphens only, no underscores):

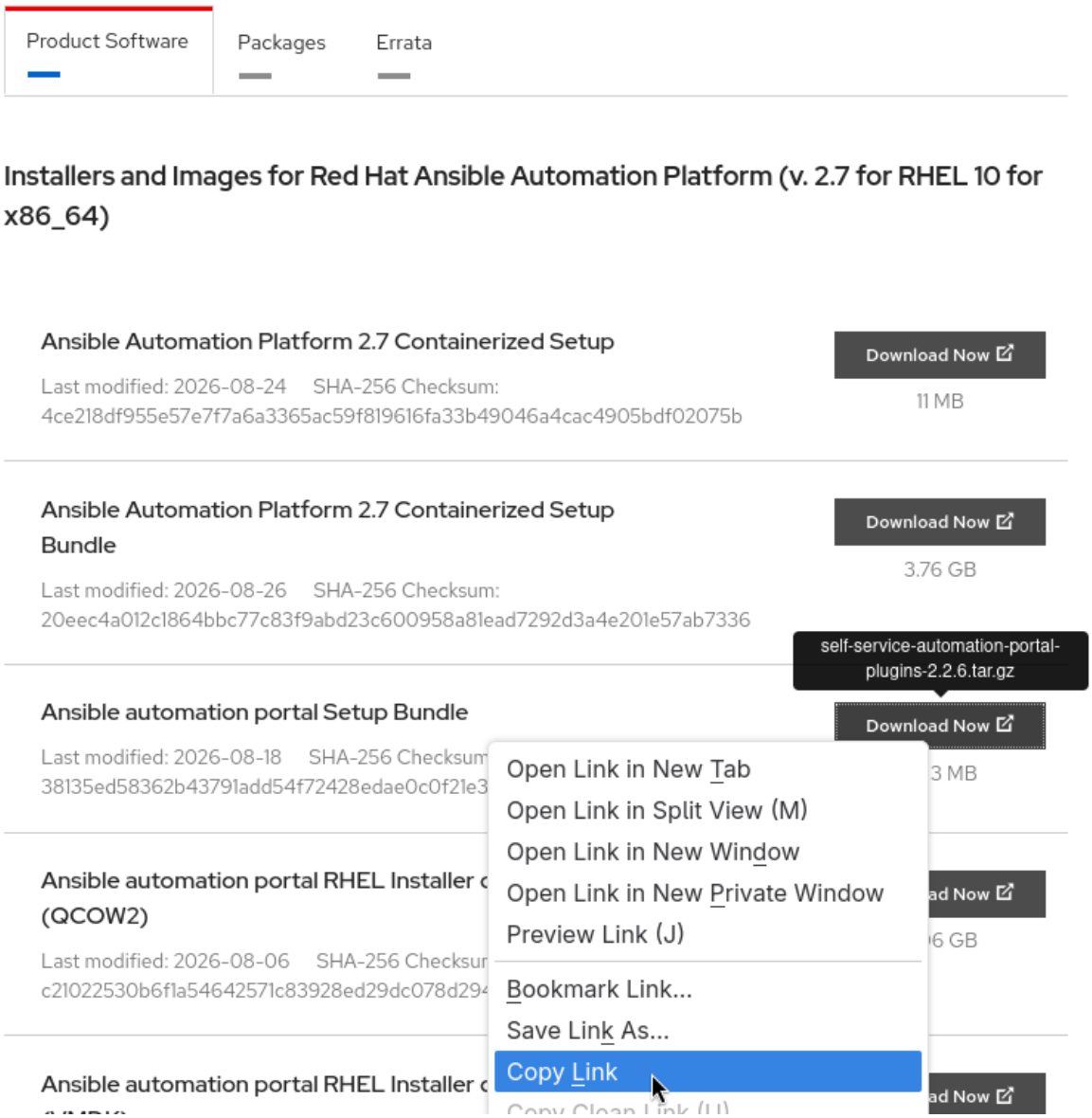
```
oc new-project portal
```

4. Make sure your CLI context is set to the newly created project:

```
oc project portal
```

5. Build and Deploy the Plugin Registry

6. Go to the Installer link and Get the installer link for **Ansible automation portal Setup Bundle**.




7. Download the installer file to the VM:

```
wget -O automation.tar.gz "<paste-link-here taken from above 5>"
```

8. Extract the **automation.tar.gz** file using the following command:

```
tar -xvf automation.tar.gz
```

   a. Run the following commands to create the `plugin-registry` binary build, build the image using the dynamic plugin directory, and deploy the resulting image:

      Where `$DYNAMIC_PLUGIN_ROOT_DIR` is the local directory containing the dynamic plugin files.

      ```
      export DYNAMIC_PLUGIN_ROOT_DIR=/home/lab-user
      ```

```
oc new-build httpd --name=plugin-registry --binary
```

```
oc start-build plugin-registry --from-dir=$DYNAMIC_PLUGIN_ROOT_DIR --wait
```

```
oc new-app --image-stream=plugin-registry
```

```
Currently, we are using Helm chart to deploy the Automation Portal. In future, we will
be using Operator to deploy the Automation Portal once its will be GA. Currently its
in in Tech Preview.
```

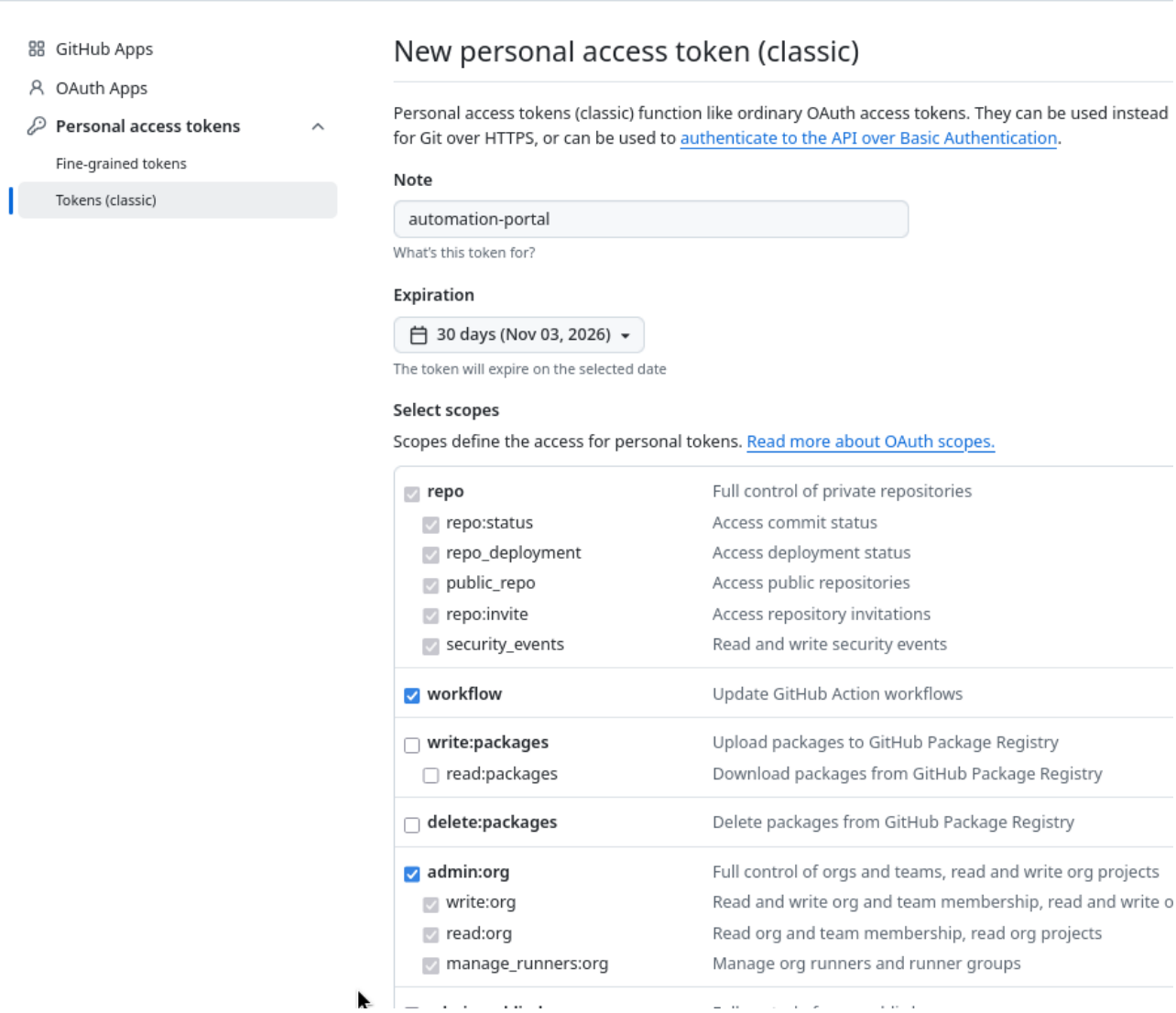
## Create a Github Token (PAT) and Oauth Application for pushing the Execution Environment (EE) to Github.

1. The Github Token (PAT) is needed to use the Content Discovery feature of the Automation Portal.
2. We are going to setup the Github OAuth Application to push the Execution Environment (EE) content to your GitHub repository and then with Github Actions we will build the EE and push it to the Red Hat registry. So for this you need to use your personal Github Account.

#### Step 1: Generate a GitHub Personal Access Token (PAT)

1. Go to your GitHub Settings > Developer Settings > Personal Access Tokens.
2. Click Generate new token (classic).
3. Give it a descriptive note (e.g., AAP-Content-Discovery) and Create a GitHub PAT with the following scopes:

```
repo
admin (this can be only read:*org*)
workflow (required for the CI activity tab to display workflow run status)
```




4. Click Generate Token and immediately copy the token value as it will be used in the next section for creating secrets in the Automation Portal.

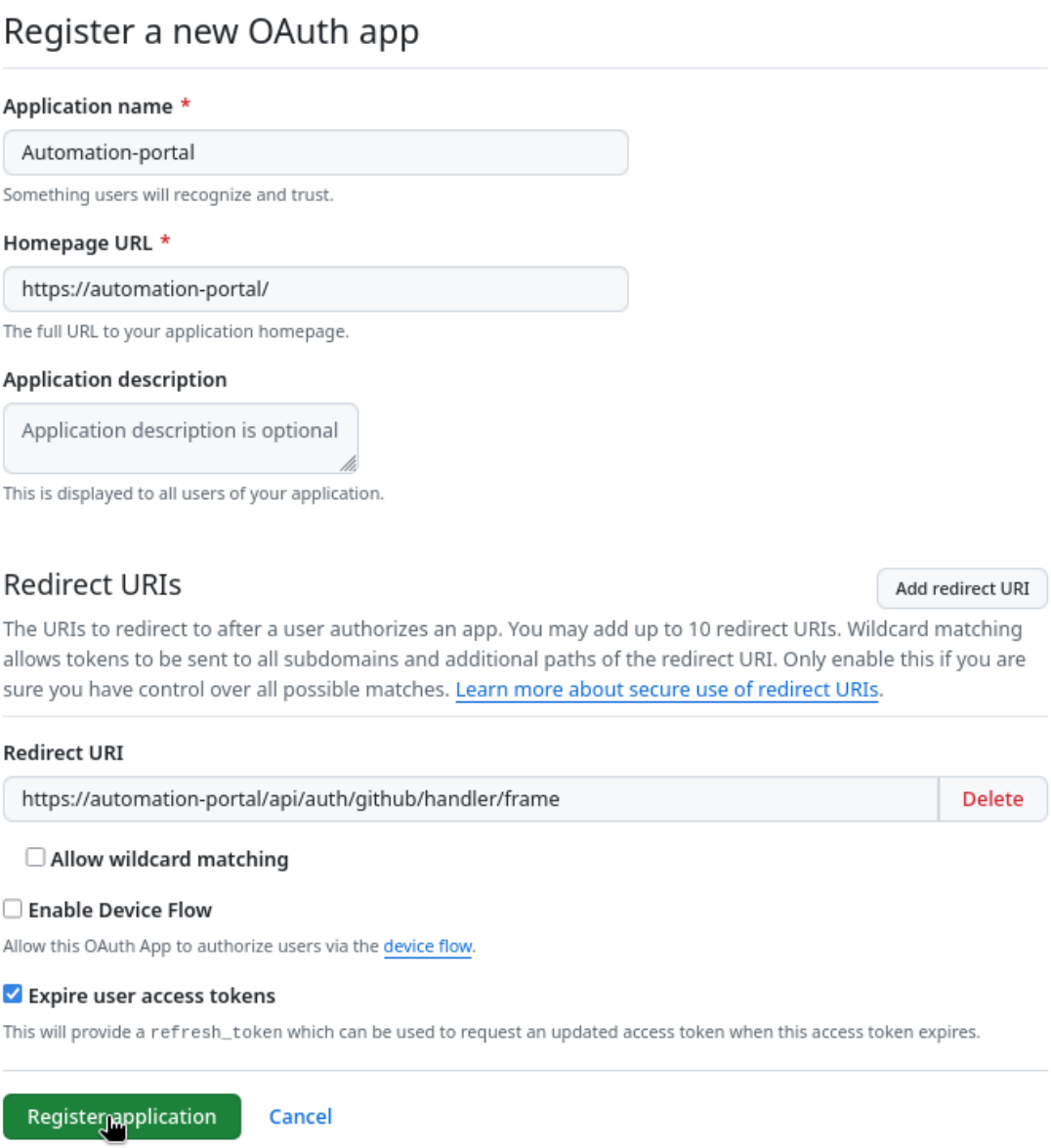
**Step 2: Create an OAuth apps in GitHub for the EE Builder.**

To configure GitHub OAuth for the automation portal, complete the following steps.

1. Create a GitHub OAuth App in your organization settings under **Developer settings** > **OAuth Apps**.

```
Enter the following details:
```

   - **Application name:** A descriptive name, for example `ansible-portal-ee-builder`.
   - **Homepage URL:** The URL of your automation portal deployment. `https://automation-portal`
   - **Authorization callback URL:** `https://automation-portal/api/auth/github/handler/frame`






GitHub limits the Authorization callback URL to 100 characters and also we will replace the placeholder `automation-portal` with the actual hostname After portal deployment. If your OpenShift route URL exceeds this limit, configure a shorter route hostname for the portal before creating the OAuth App.

If your OpenShift route URL exceeds this limit, configure a shorter route hostname for the portal before creating the OAuth App.

2. Note the **Client ID** and generate a **Client secret**.

```
Save the client secret value immediately because you cannot view it again.
```

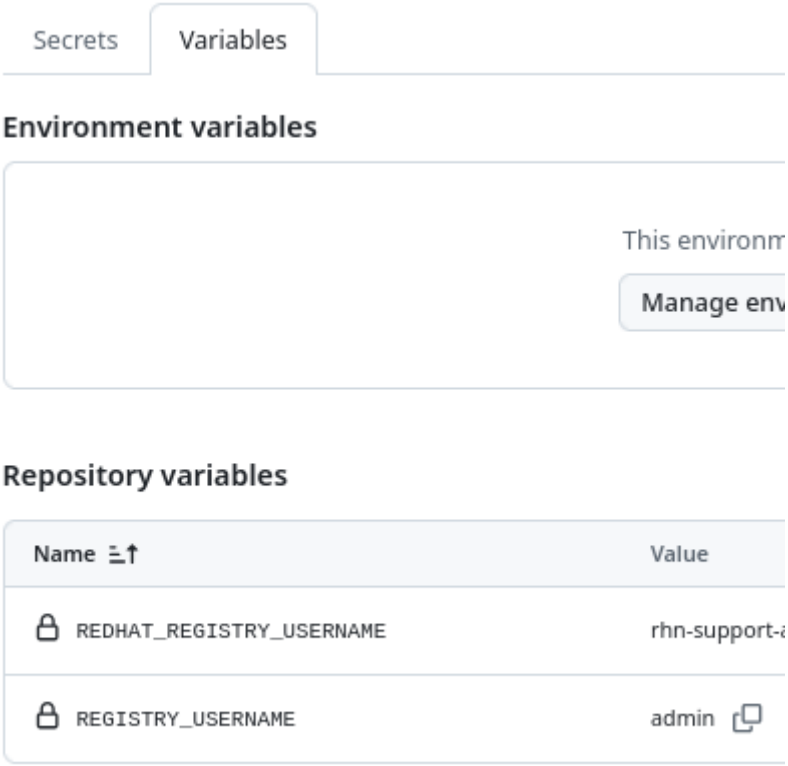
3. If your organization restricts OAuth App access, navigate to **Organization settings** > **Third-party access** and approve the OAuth App.
4. we will Enable the GitHub authentication provider in your configuration in the next steps.

**Step 3: Create a repo:**

In your GitHub organization which we will be integrating in the next chapter please store the Execution Environment (EE) content. This repository will be used by the Automation Portal to push and manage EE content.

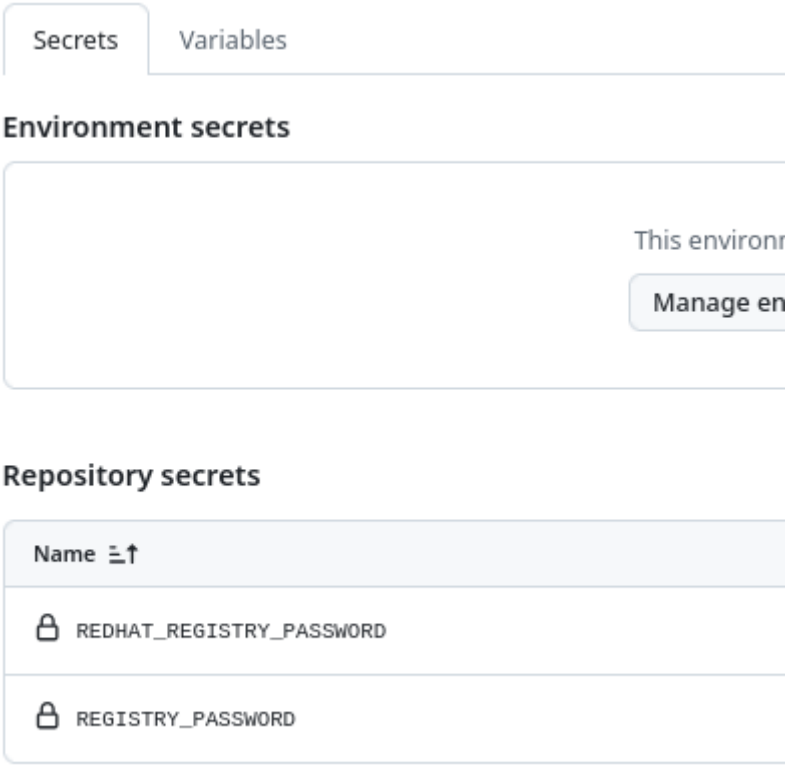
1. I have created automation-portal you can create your own repo in your GitHub organization to store the Execution Environment (EE) content. This repository will be used by the Automation Portal to push and manage EE content.
2. Go to settings of the repository and then go to secrets and variables and then click on actions and then click on new repository secret.

```
# For Variables
REDHAT_REGISTRY_USERNAME: <YOUR-RHN-ID-USERNAME>
REGISTRY_USERNAME : admin
```




```
# For Secrets

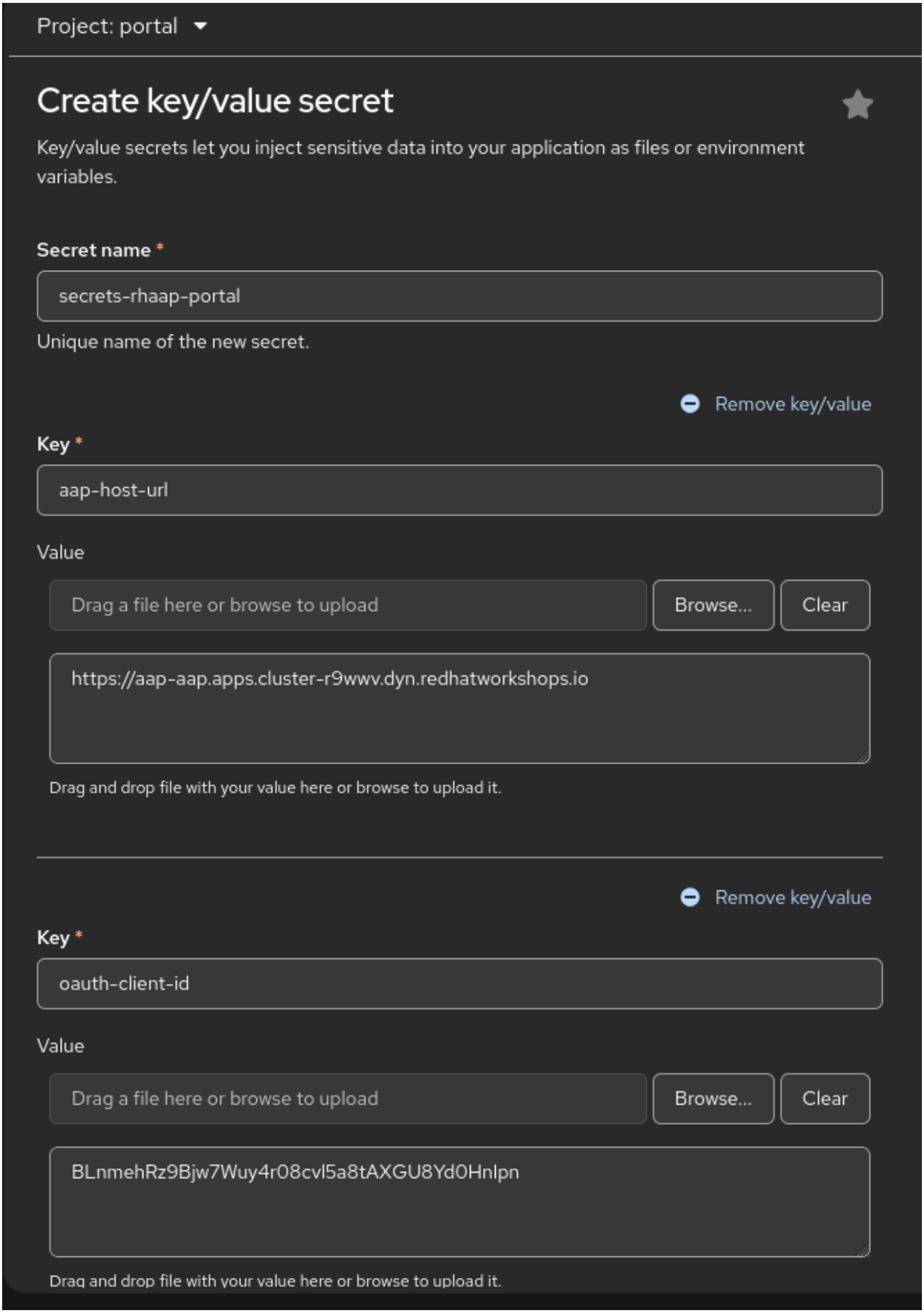
REDHAT_REGISTRY_PASSWORD: <YOUR-RHN-ID-PASSWORD>
REGISTRY_PASSWORD : <RHDP-AAP-Admin-Password>
```

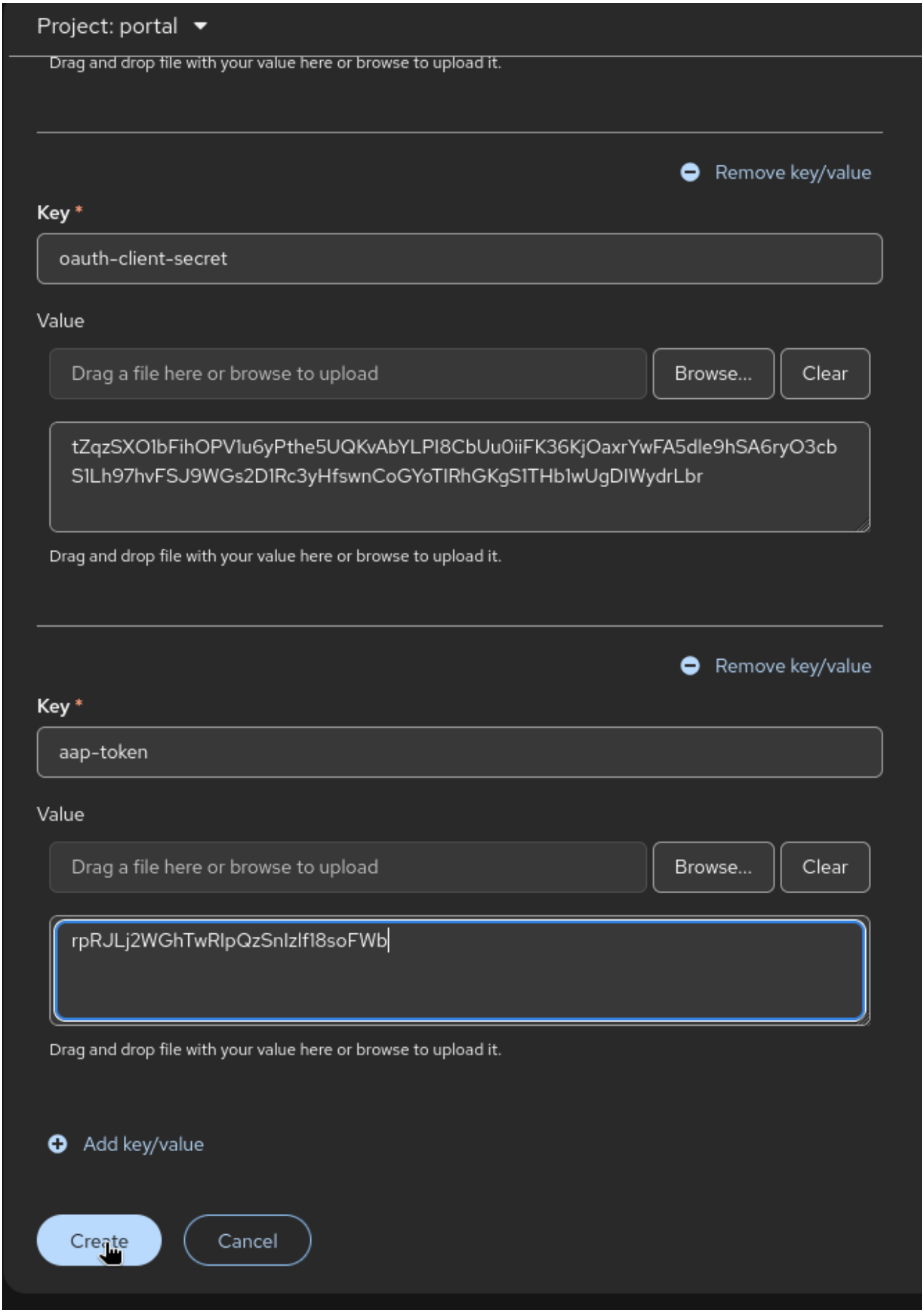



## Create OpenShift Secrets for AAP Authentication

OpenShift Secrets are secure, encrypted storage for sensitive data. We'll create a secret that holds the OAuth credentials and API token you generated in Tasks 1 and 3. The Automation Portal Helm chart will reference this secret during deployment.

1. Go to Workloads > Secrets in the OpenShift console and make sure the project `portal` is selected. You should create the secret `secrets-rhaap-portal` with the below information. You can also create the secret using the `oc` command line tool.


Project: portal
Create key/value secret
Key/value secrets let you inject sensitive data into your application as files or environment variables.
Secret name *
secrets-rhaap-portal
Unique name of the new secret.
Remove key/value
Key *
aap-host-url
Value
Drag a file here or browse to upload
Browse...
Clear
https://aap-aap.apps.cluster-r9wwv.dyn.redhatworkshops.io
Drag and drop file with your value here or browse to upload it.
Remove key/value
Key *
oauth-client-id
Value
Drag a file here or browse to upload
Browse...
Clear
BLnmehRz9Bjw7Wuy4r08cvl5a8tAXGU8Yd0Hnlpn
Drag and drop file with your value here or browse to upload it.




OR

You can also run the secret creation command, gather the values from Tasks 1 and 3:

- `<aap-instance-url>` — The URL of your AAP instance (e.g., `https://{aap_controller_web_url}`)
- `<oauth-client-id>` — The Client ID from the previous task.
- `<oauth-client-secret>` — The Client Secret from the previous task.
- `<aap-token>` — The API Token from the previous task.

1. Run the following command, replacing the placeholders with your actual values:

```
oc create secret generic secrets-rhaap-portal \
  --namespace portal \
  --from-literal=aap-host-url="<your-aap-url>" \
  --from-literal=oauth-client-id="your-client-id-here" \
  --from-literal=oauth-client-secret="your-client-secret-here" \
  --from-literal=aap-token="your-api-token-here" \
  -n portal
```

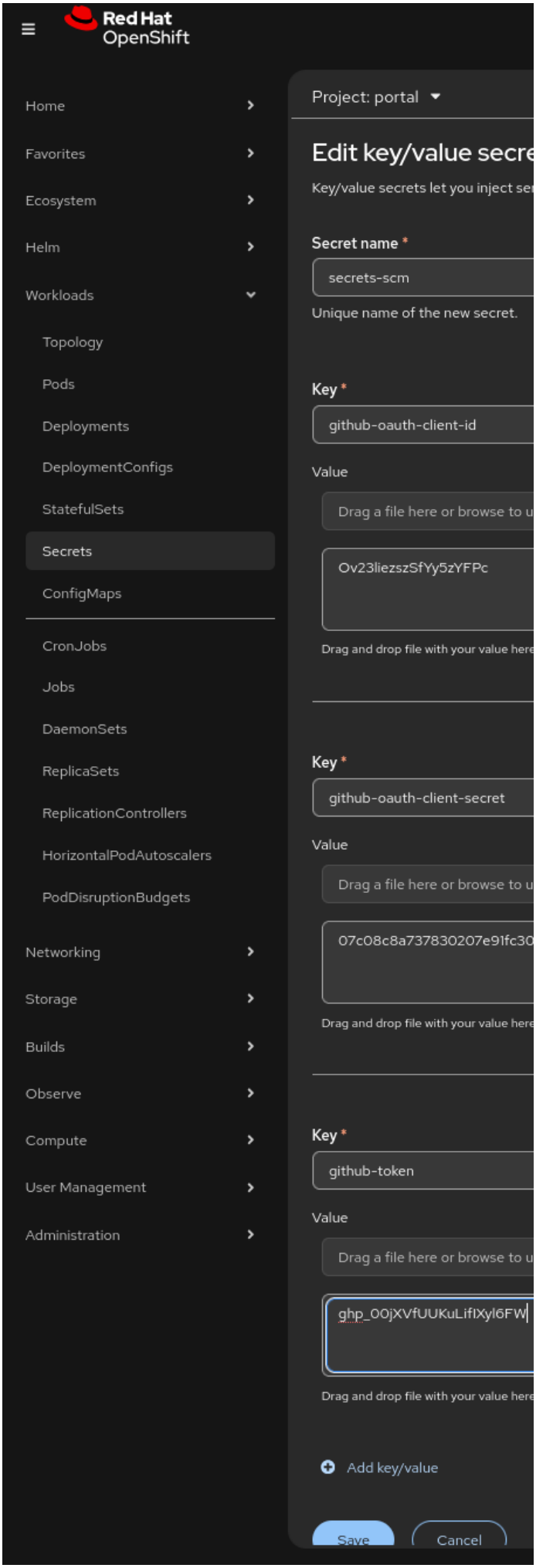


The secret name (`secrets-rhaap-portal`) is important—the Helm chart expects this exact name. If you change it, you'll need to update your Helm values during deployment.

2. Create a secret for GitHub OAuth and Personal Access Token (PAT) in the same way, using the values from Task 4. This secret will be used by the Automation Portal to authenticate with GitHub for pushing Execution Environment content.

   a. You can create using the below command or you can create it from the OpenShift console > Workloads > Secrets in the project `portal` . You should create the secret `secrets-scm` with the below information.

```
oc create secret generic secrets-scm \
 --namespace portal \
 --from-literal=github-token="<GITHUB-PAT-TOKEN>" \
 --from-literal=github-oauth-client-id="<GITHUB-OAUTH-CLIENT-ID>" \
 --from-literal=github-oauth-client-secret="<GITHUB-OAUTH-CLIENT-SECRET>" \
```


Red Hat OpenShift
Home
Favorites
Ecosystem
Helm
Workloads
Topology
Pods
Deployments
DeploymentConfigs
StatefulSets
Secrets
ConfigMaps
CronJobs
Jobs
DaemonSets
ReplicaSets
ReplicationControllers
HorizontalPodAutoscalers
PodDisruptionBudgets
Networking
Storage
Builds
Observe
Compute
User Management
Administration
Project: portal
Edit key/value secre
Key/value secrets let you inject se
Secret name *
secrets-scm
Unique name of the new secret.
Key *
github-oauth-client-id
Value
Drag a file here or browse to u
Ov23liezszSfYy5zYFPc
Drag and drop file with your value here
Key *
github-oauth-client-secret
Value
Drag a file here or browse to u
07c08c8a737830207e91fc30
Drag and drop file with your value here
Key *
github-token
Value
Drag a file here or browse to u
ghp_00jXVfUUKuLifIXyl6FW
Drag and drop file with your value here
Add key/value
Save
Cancel

Once the above is done. You're now ready for the next phase: deploying the Automation Portal via Helm chart!

# Chapter 4: Synchronizing Red Hat Certified Automation Content

In this lab, you will configure your Private Automation Hub to sync Red Hat Certified Content collections directly from `cloud.redhat.com` (Console Red Hat).

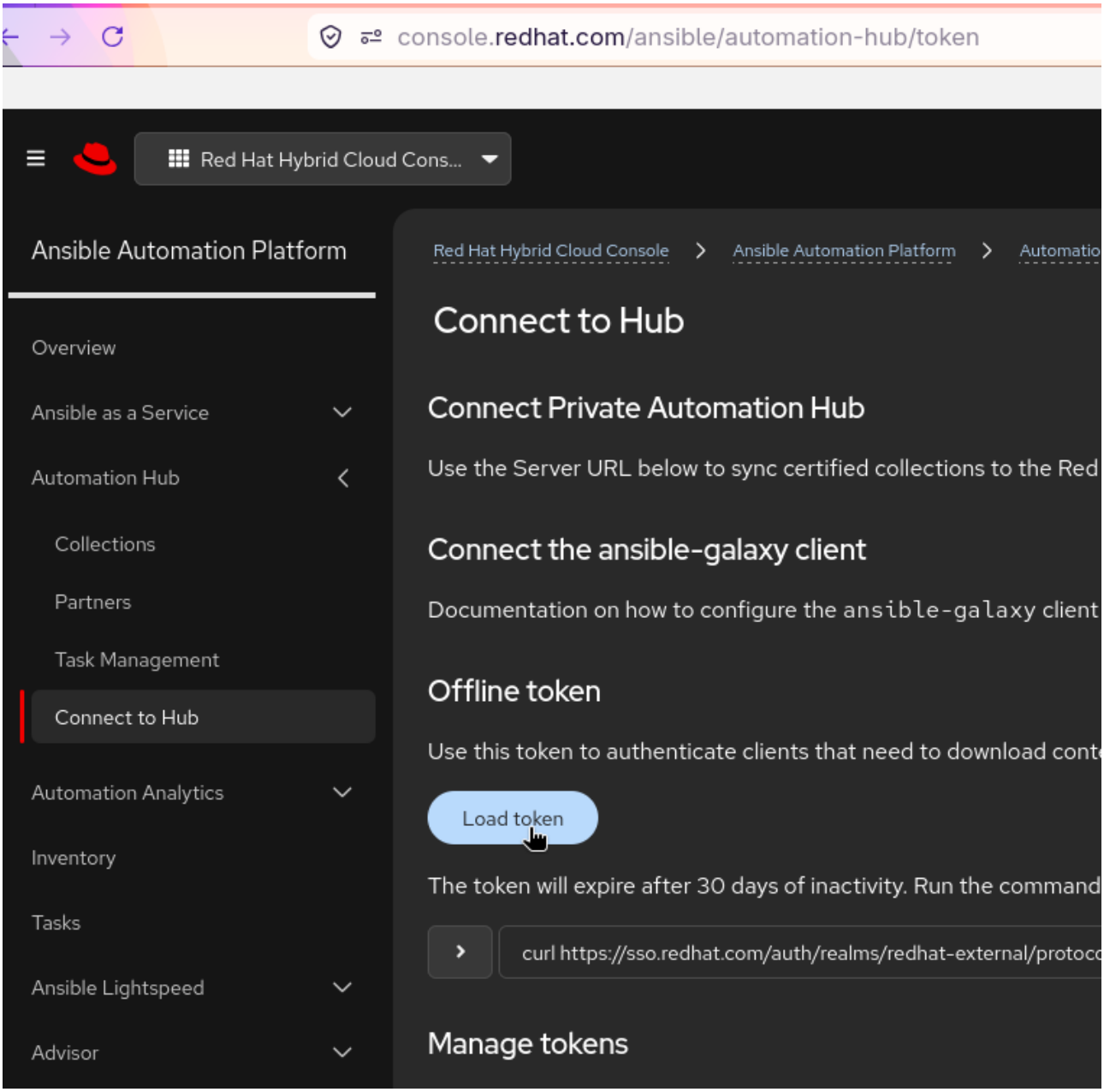


In most enterprise production environments, this integration is pre-configured by system administrators during deployment. In this lab environment, you will walk through obtaining the required API token and setting up the remote repository manually to understand how the synchronization mechanism works.

## Step 1: Obtain the API Token from Red Hat Cloud

To allow Private Automation Hub to securely pull content from Red Hat's hosted Automation Hub, you need an API authentication token.

1. Open a web browser and navigate to cloud.redhat.com.
2. Log in with your Red Hat credentials.
3. Navigate to **Ansible Automation Platform** > **Automation Hub** > **Connect to Hub**.
4. Under the **Offline Token** / **Load Token** section, click **Get Token** (or **Load Token**).

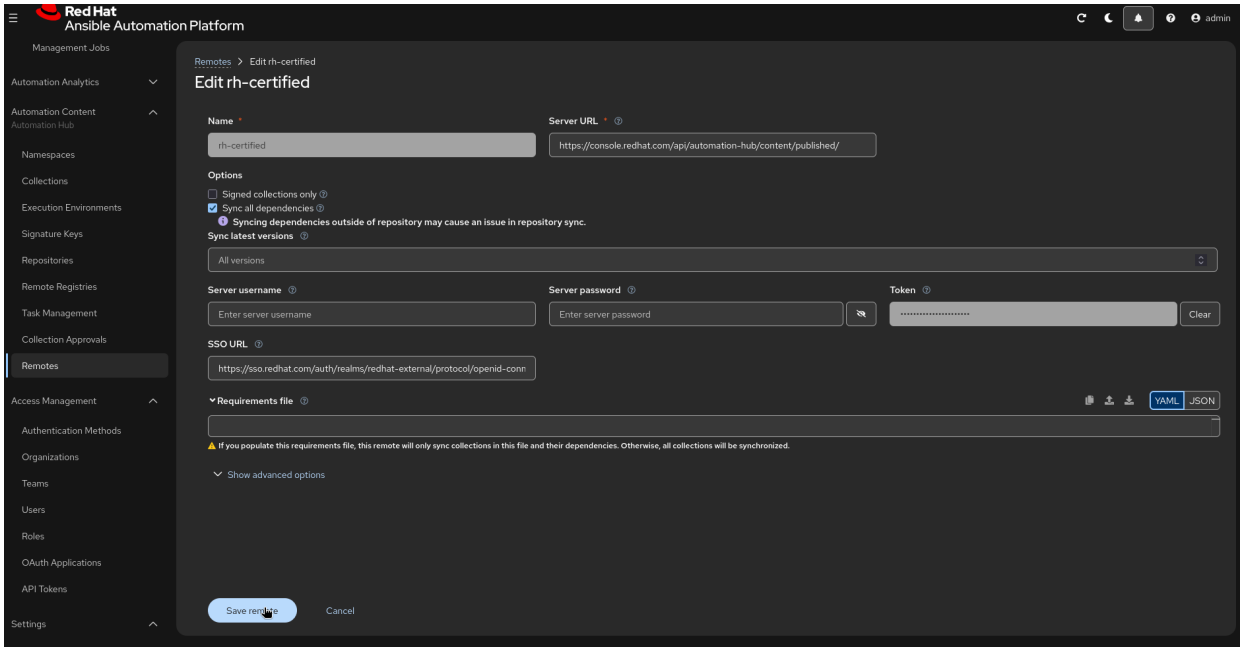



5. Copy the generated API token to your clipboard for use in the next step.

## Step 2: Configure the Remote Repository in Private Automation Hub

Now that you have the API token, update the remote repository configuration on your local Private Automation Hub instance.

1. Open a new browser tab and log in to your **Automation Content** interface as an administrator.
2. In the left navigation menu, expand **Repository Management** and select **Remote**.
3. Locate the `rh-certified` remote entry in the repository list, click the **Options** menu (⋮) next to it, and select **Edit**.
4. Update the following fields:
   - **Token:** Paste the API token you copied from `cloud.redhat.com` in Step 1.

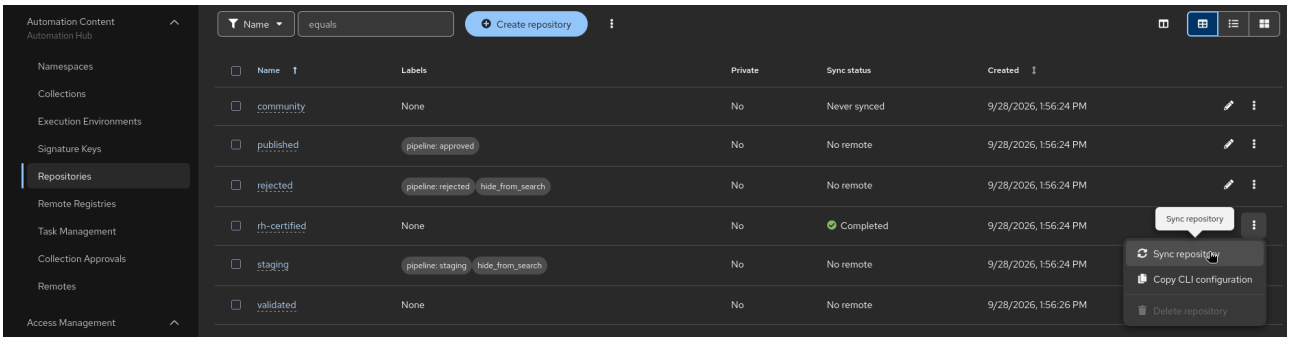



5. Click **Save**.

## Step 3: Trigger Content Synchronization

Once the token and URL are configured, pull the certified collection metadata and artifacts into your local environment.

1. On the **Remotes** page, locate the `rh-certified` repository again.
2. Click the **Options** menu (⋮) and select **Sync**.



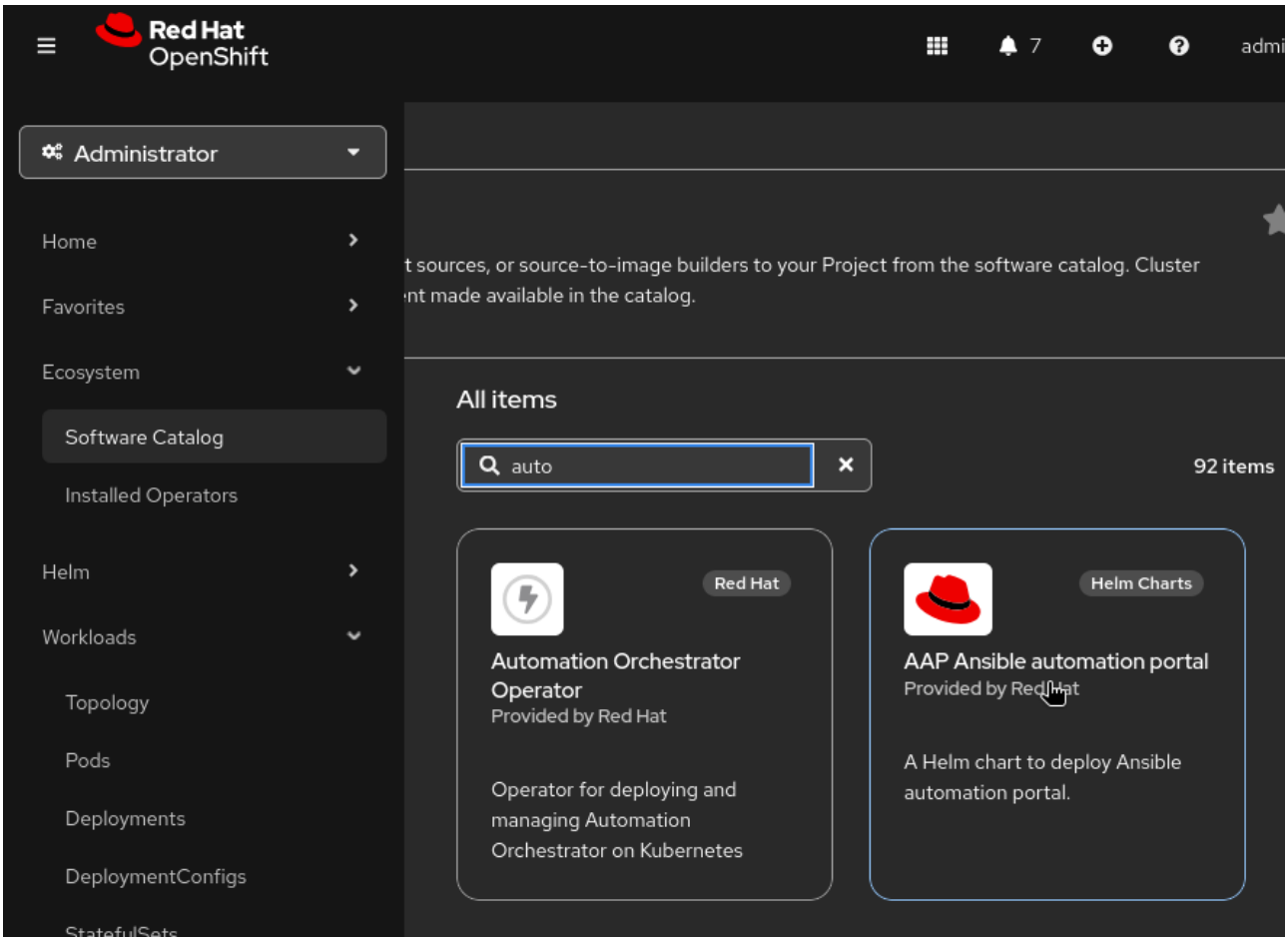

3. Navigate to **Task Management** (under Administration) to monitor the progress of the synchronization job.
4. Once the status displays **Completed**, navigate to **Collections** > **Red Hat Certified** in the navigation menu to verify that certified content collections are now populated and accessible locally.

# Hands-on Lab: Installing and Verifying the Automation Portal on OCP

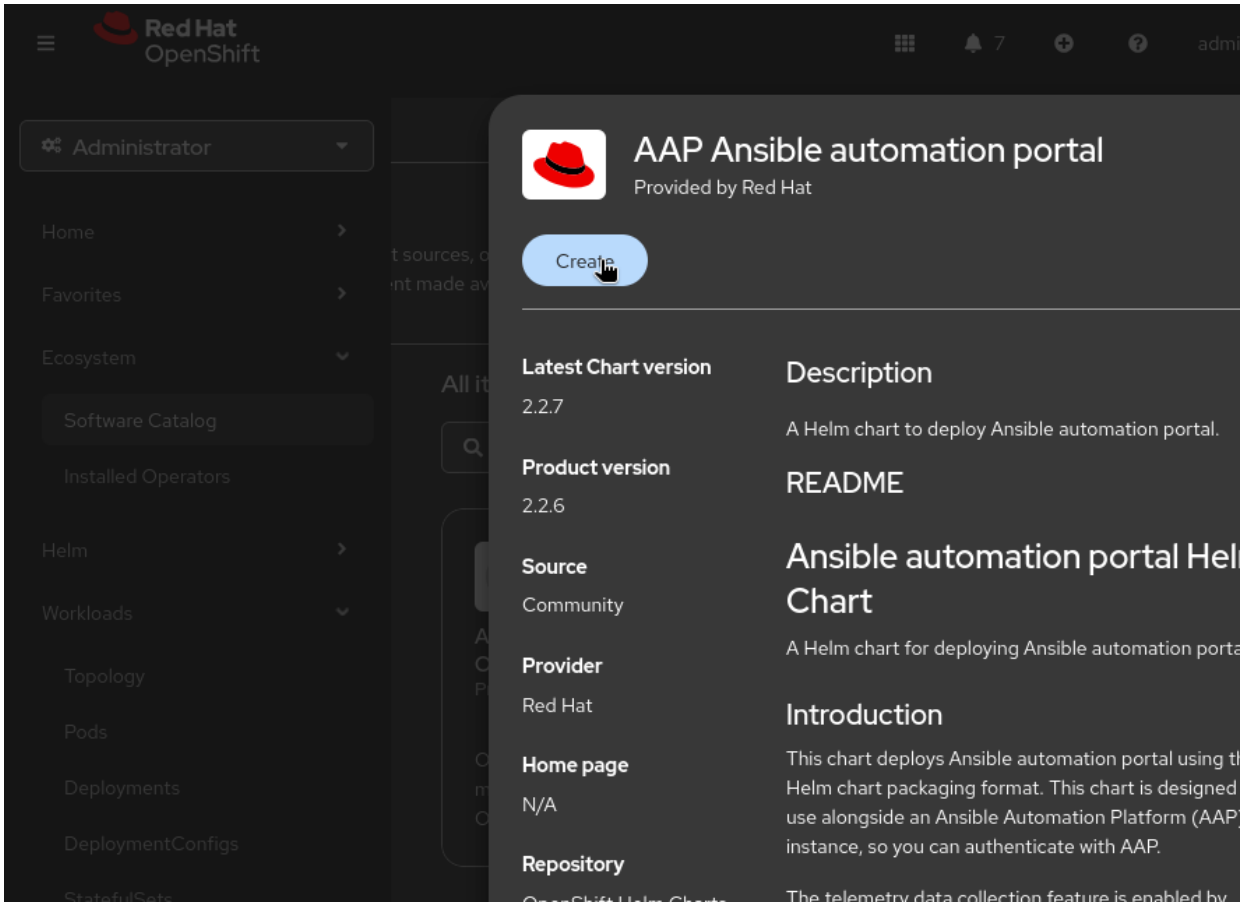
Now we are all set to do the deployment of the Automation Portal on OpenShift Container Platform (OCP) and CI/CD for the EE builder that we are going to learn in the Next Chapter.

## Installing the Automation Portal

1. Login to the Openshift Cluster and Ansible Automation Platform (AAP) as admin user from the RHDP login page credentials.
2. Go to Ecosystem > Software Catalog in the OpenShift web console.
3. Search for "Automation Portal" with tag `Helm Chart` in the catalog and select it.

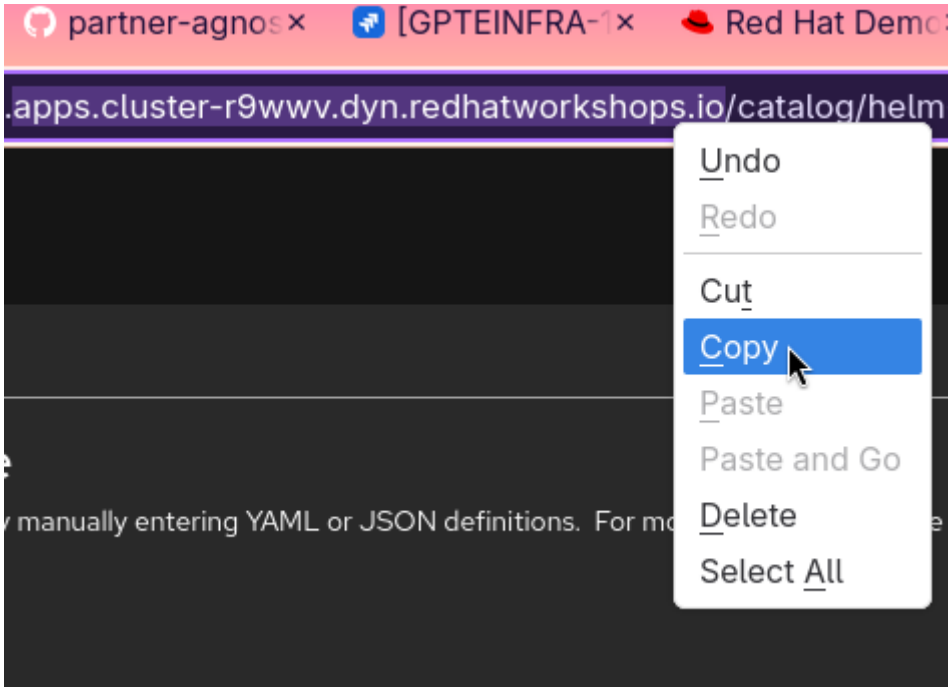


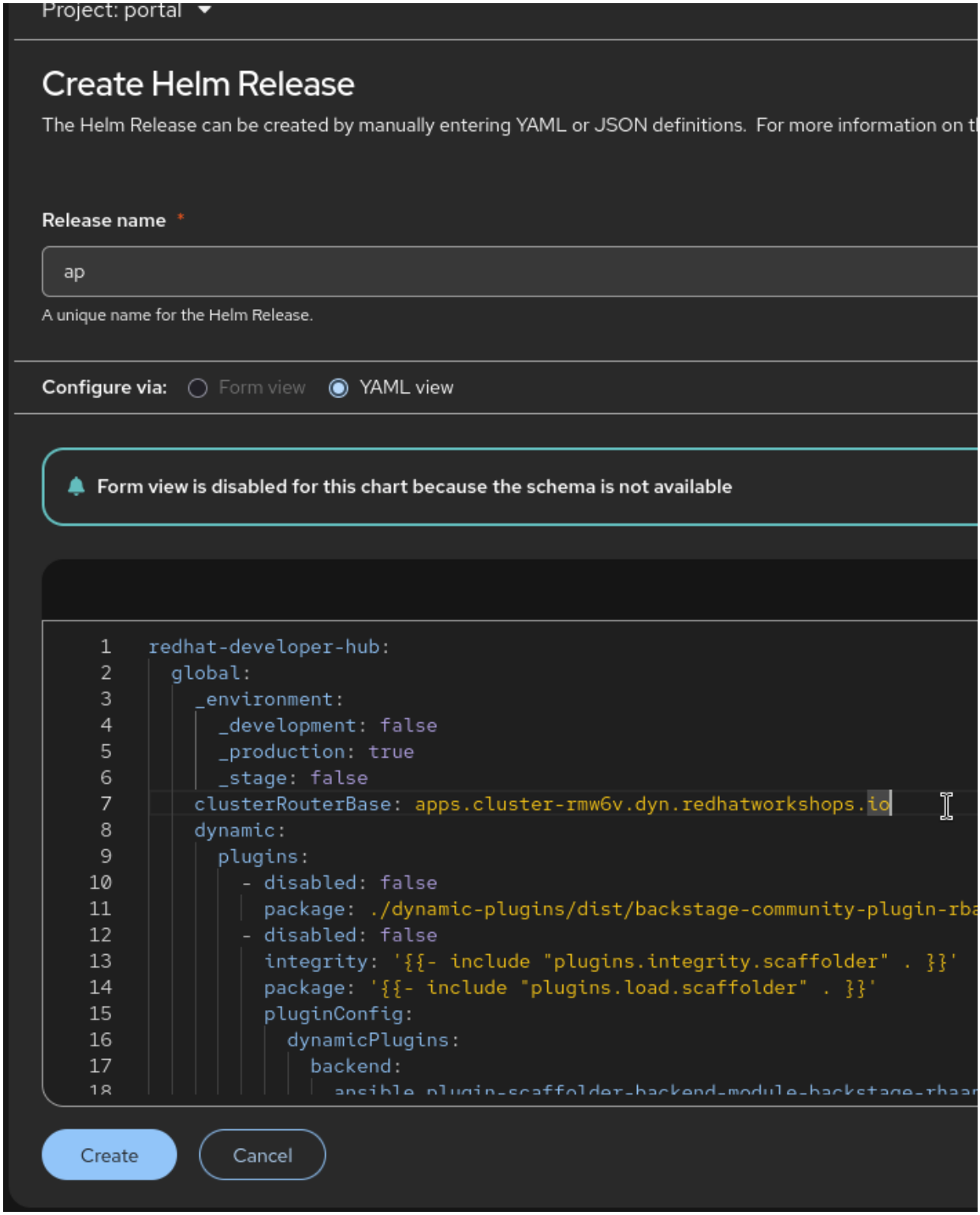

a. Click on **Create** to start the installation process.




b. Change the Release Name to `ap` and you can change it if you want.

c. Change the **clusterRouterBase** to the correct value for your cluster. For example, if the OCP cluster url is `https://console-openshift-console.apps.cluster-kstzt.dyn.redhatworkshops.io` then the clusterRouterBase should be `apps.cluster-kstzt.dyn.redhatworkshops.io`.

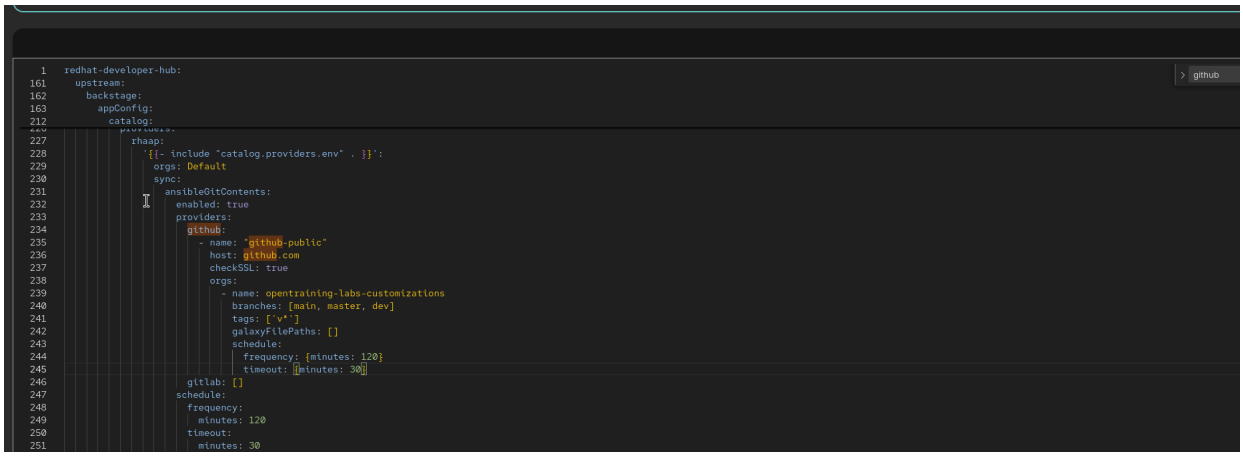




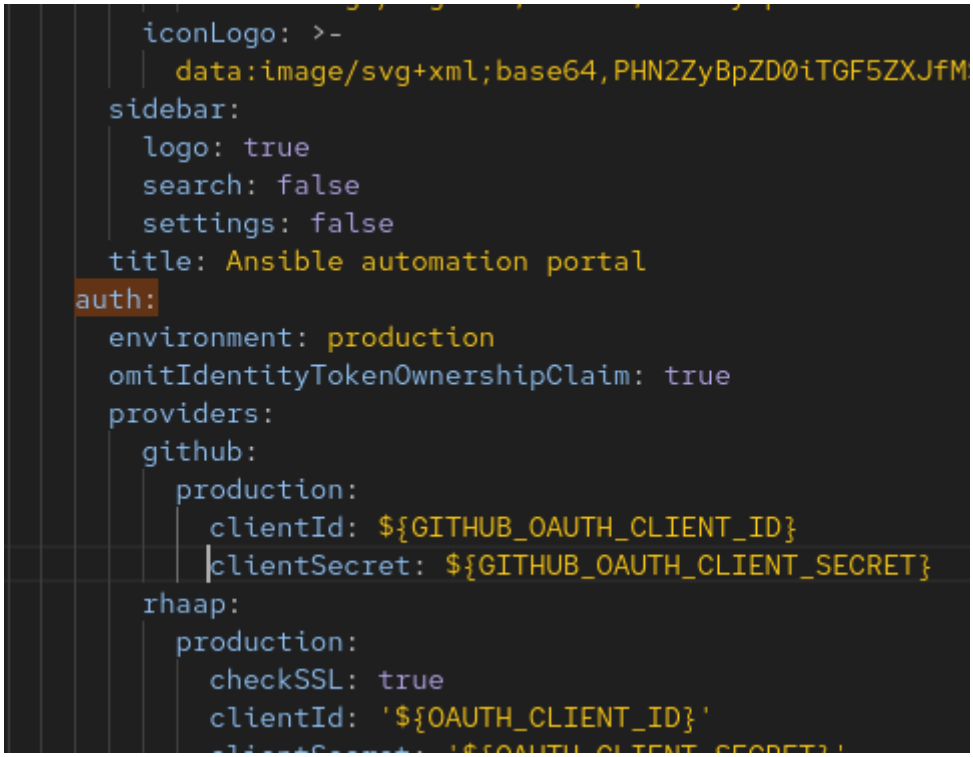

d. In addition to the above settings, we will configure the GIT and the PAH to enable the Automation Portal to push the Execution Environment content to the GIT repository and also to pull the certified content from the Private Automation Hub.

```
ansibleGitContents:
                  enabled: true  <-- this needs to be true to enable the git
content push to the git repository.
                  providers:
                    github:
                      - name: "github-public"
```

```
                          host: github.com  <-- my github host is github.com,
you can change it to your github host if you are using a private github
instance.
                          checkSSL: true
                          orgs:
                            - name: <REPLACE_WITH_YOUR_GITHUB_ORGANIZATION> <--
replace this with your github organization name where you want to push the
content.
                              branches: [main, master, dev]
                              tags: ['v*']
                              galaxyFilePaths: []
                              schedule:
                                frequency: {minutes: 120}
                                timeout: {minutes: 30}
```



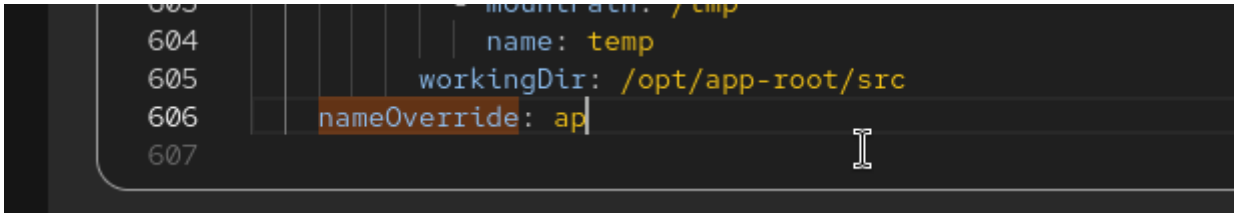

```
# The auth will be taking the GITHUB_OAUTH_CLIENT_ID and
GITHUB_OAUTH_CLIENT_SECRET from the Secrets we created in the previous section.
 auth:
  providers:
    github:
      production:
        clientId: ${GITHUB_OAUTH_CLIENT_ID}
        clientSecret: ${GITHUB_OAUTH_CLIENT_SECRET}
```

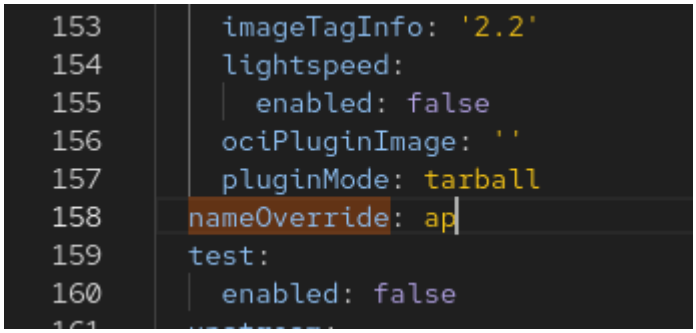



```
pahCollections:
  enabled: true <-- make sure this is true to enable the PAH collections to be
pulled from the private automation hub.
```




e. Where ever you find `nameoverride` in the configuration, you can change it to ap as a value because for github configuration we need to have the URL under 100 characters. so we are only reducing the `nameoverride` to ap to have a shorter URL.



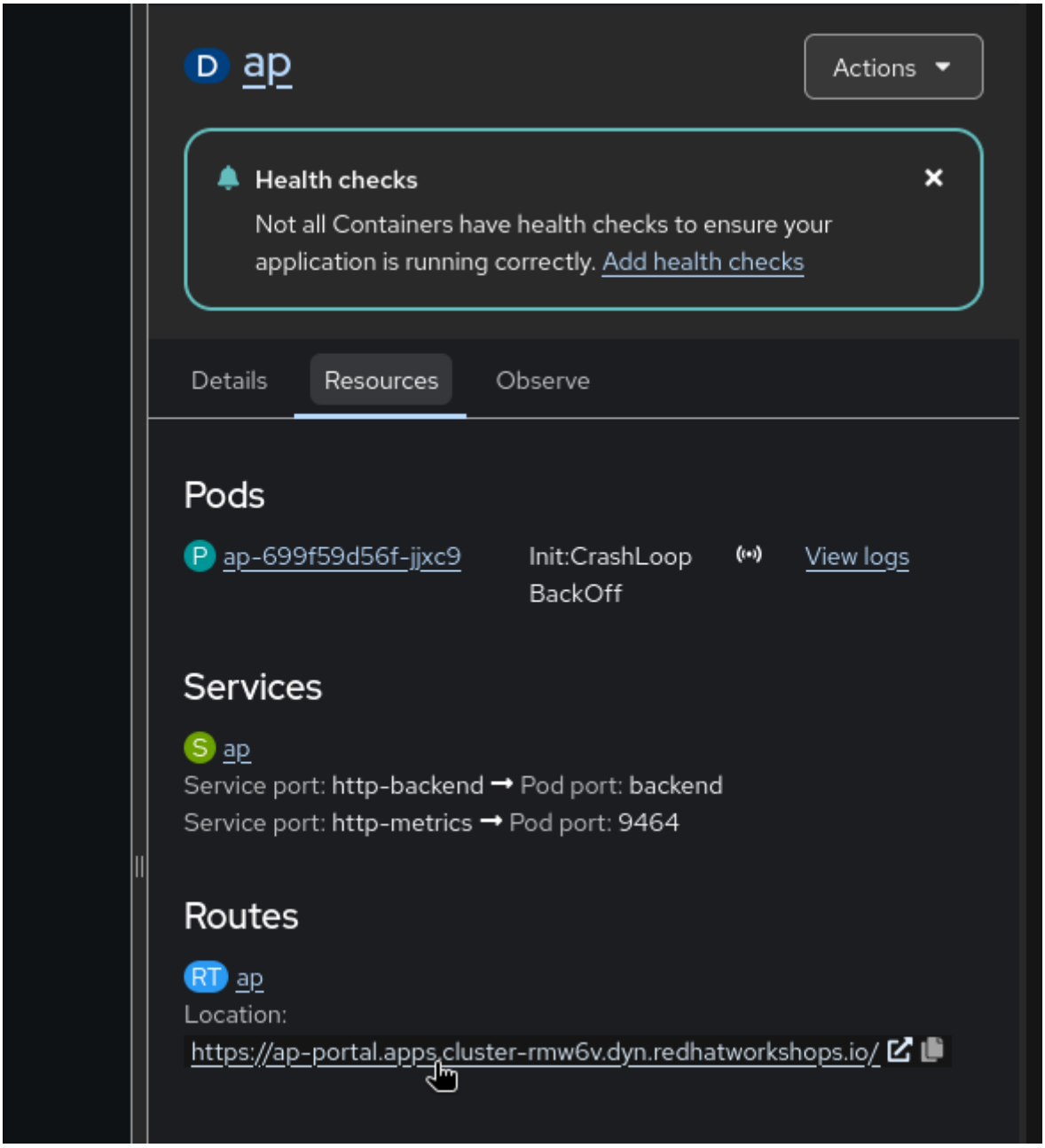




f. By default, it will be setting up at the Organization **Default** for the automaton portal the parameter is orgs if you wanted to change it.

Once the above settings are done, click on **Create** to start the installation process.

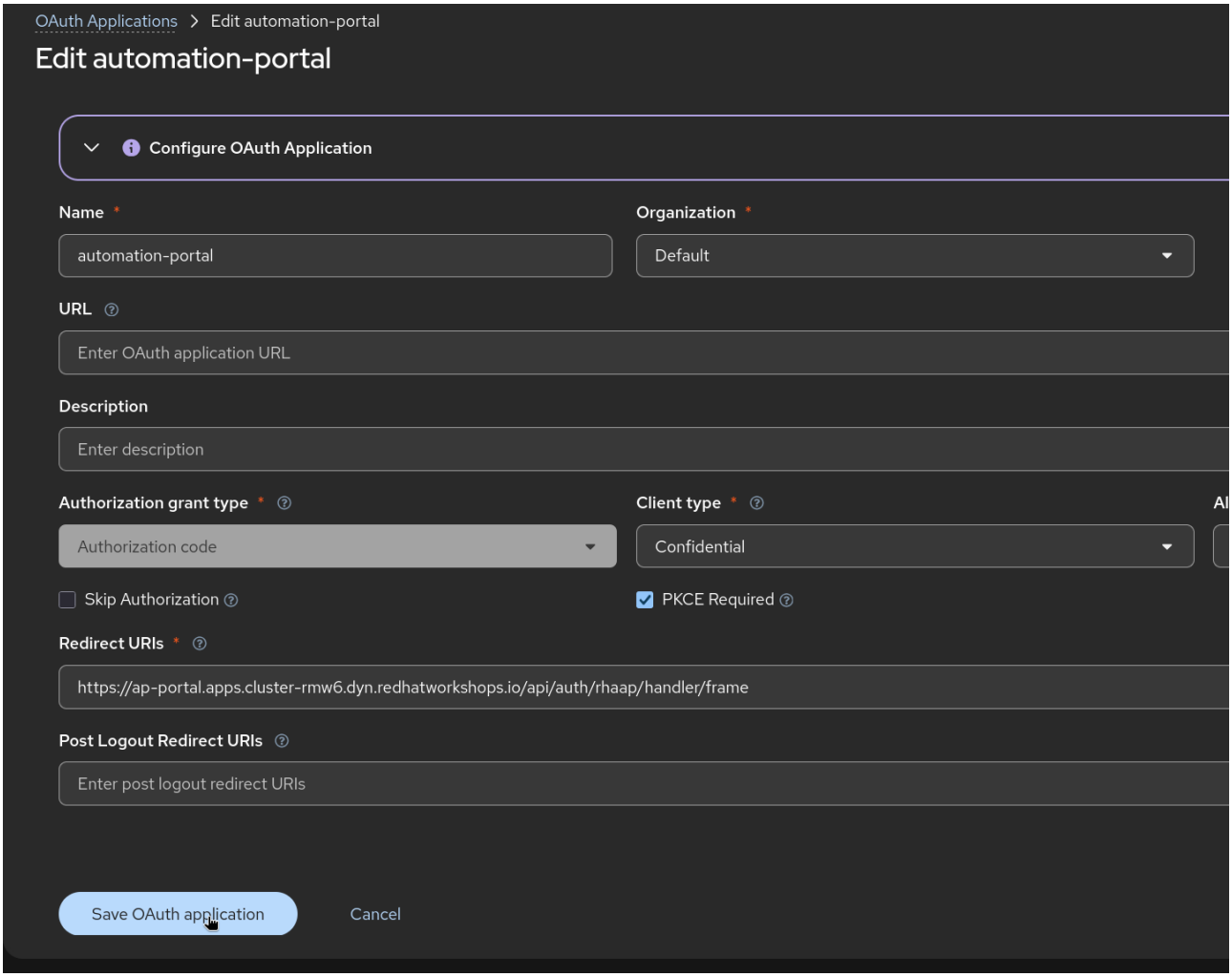
> The installation process will take a few minutes. You can monitor the progress Workloads > Topology in the portal project. Once the installation is complete, you will see the Automation Portal components running.

4. Go to Workloads > Topology in the portal project. Once the installation is complete, you will see the Automation Portal components running.
5. Click on **automation-portal-rhaap-portal** resource to view the details. You will find the route URL in the **Routes** section. This URL is used to access the Automation Portal web interface.

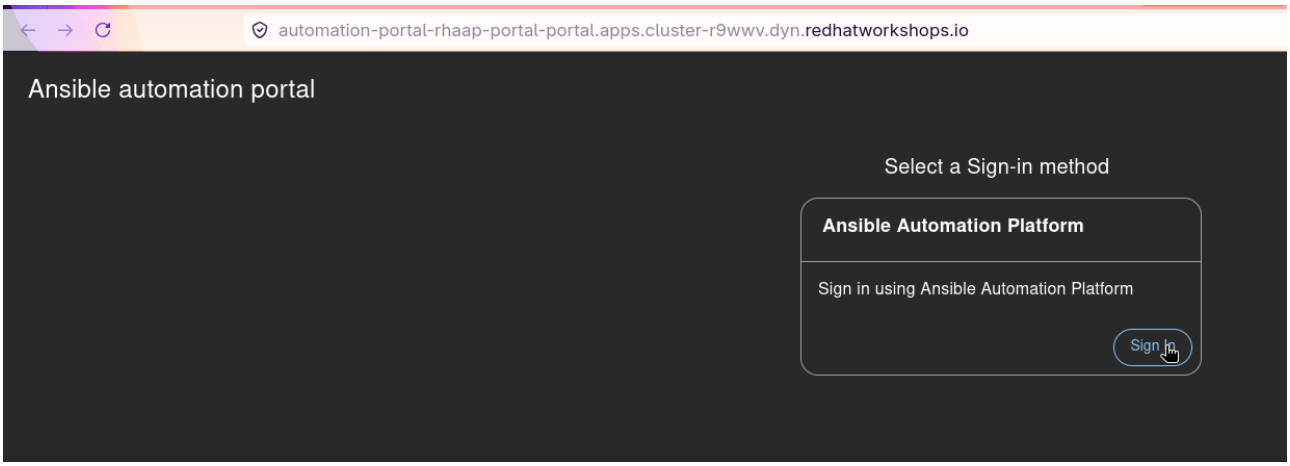



6. Copy the route URL and go to Ansible Automation Platform > Access Management > OAuth Applications.
7. Edit the automation-portal OAuth application to include the route URL as a valid redirect URI and save the changes. This step is crucial for the OAuth authentication to work correctly.

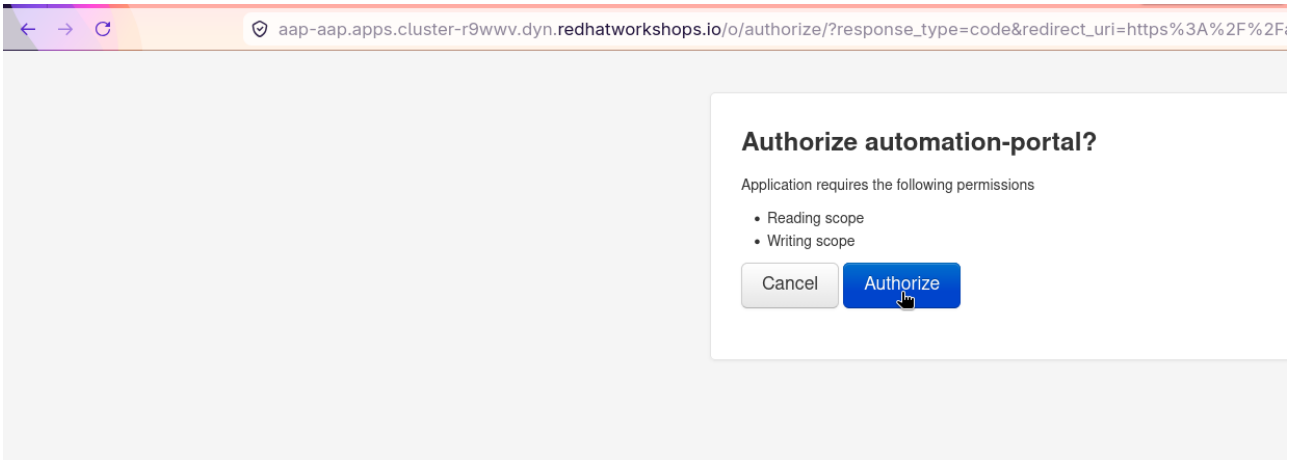
```
https://<aap-portal-hostname>/api/auth/rhaap/handler/frame
```



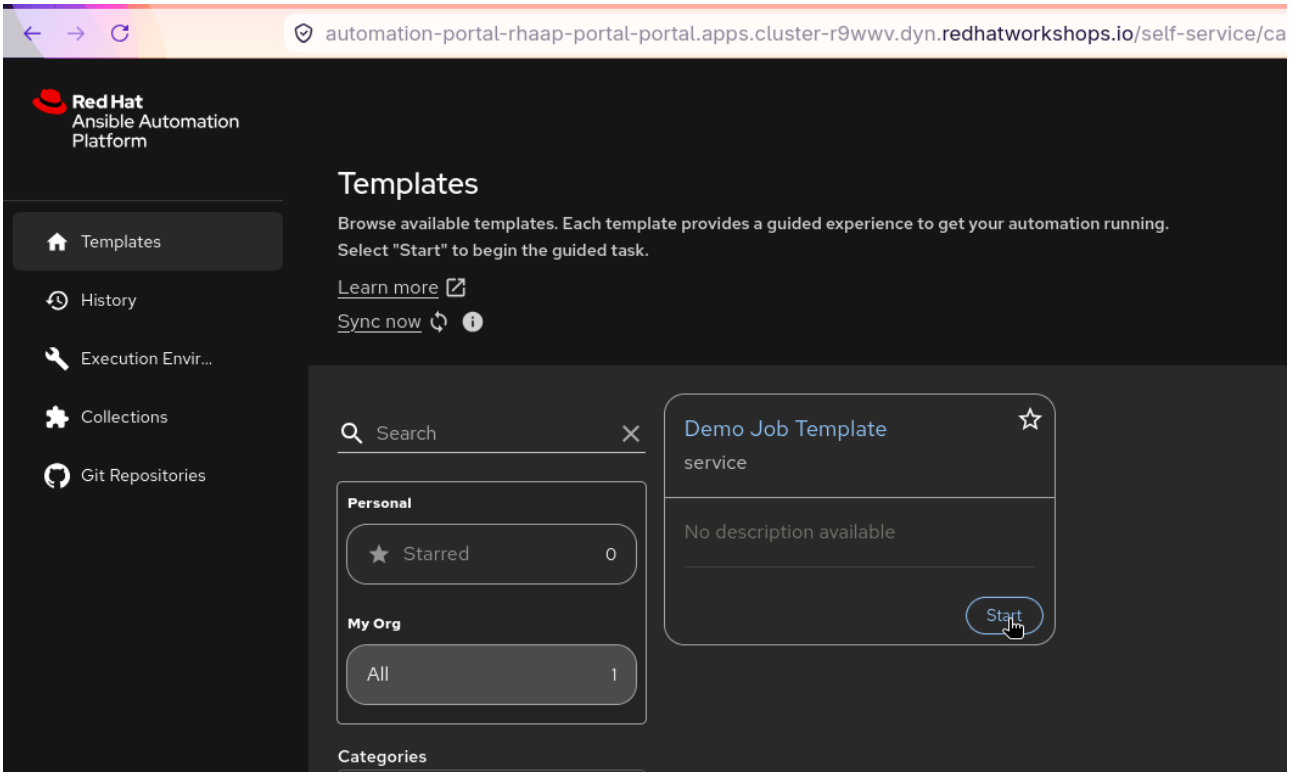

8. Access the Web UI of the Automation portal and click on the **Login with Ansible Automation Platform** button. You will be redirected to the AAP login page. Enter your AAP credentials to log in.

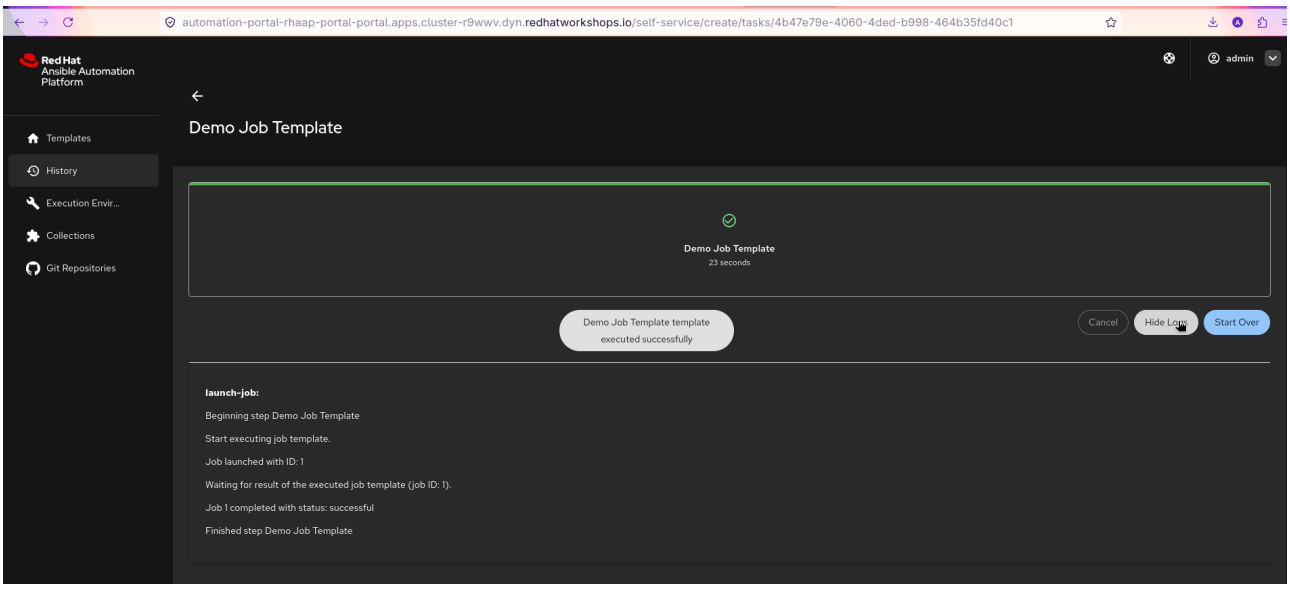



9. Once logged in, you need to provide the necessary permissions. click authorize and you will see the Automation Portal dashboard.




10. Click on the Demo Job Template and click on **Launch** to execute the job. You will see the job execution status and output.



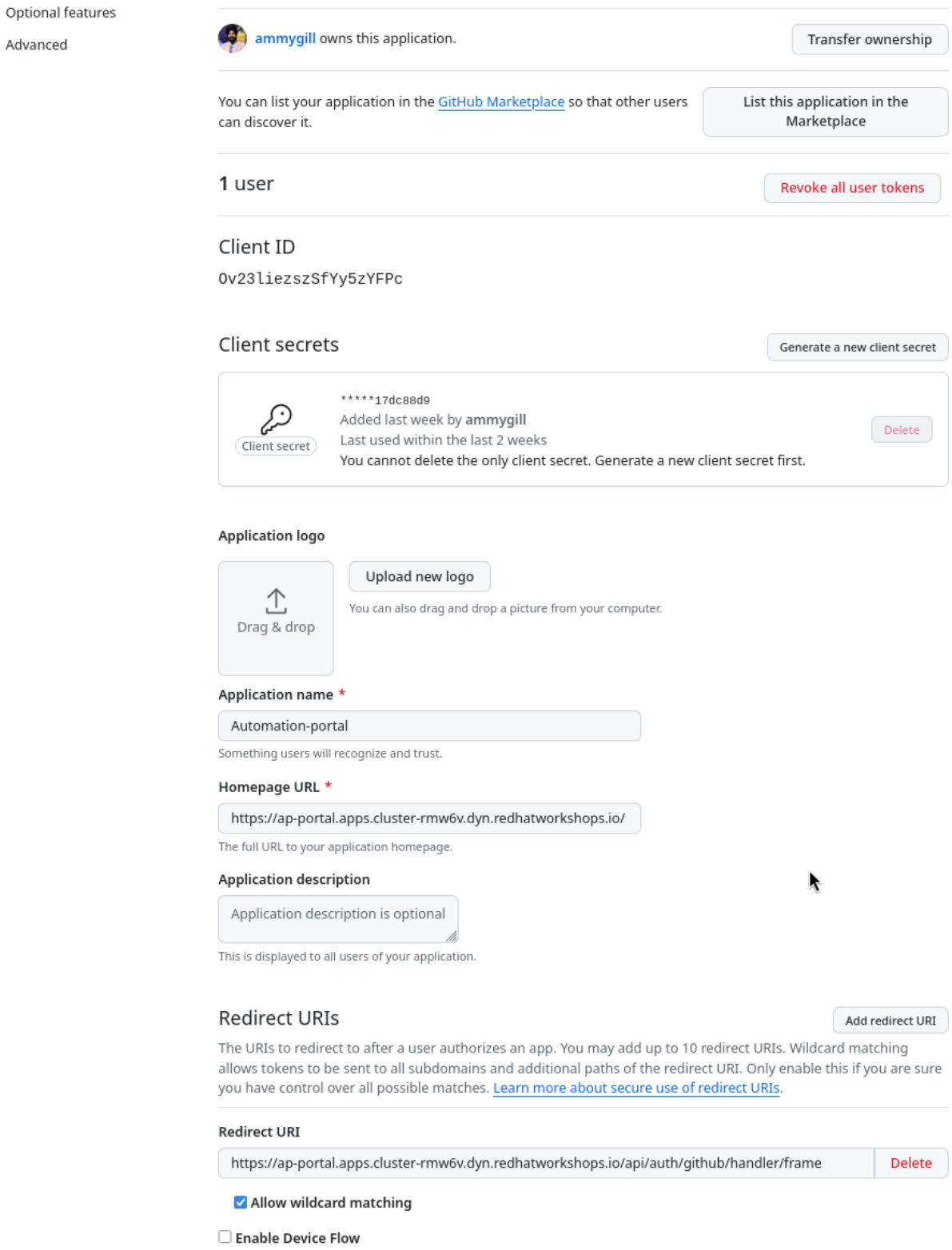
This is All about setting up the Automation Portal on OCP and verifying its functionality. You can now explore the features of the Automation Portal like RBAC Control and the execution of job templates and workflows via specific users.

For more Information on the Automation Portal or for RBAC hands on lab, refer to the Introduction to AAP Self-Service Automation

# Final Configuration of the Execution Environment

To enable the Execution Environment Builder to push content to the Git repository, we need to configure and fine-tune the OAuth applications created in Git.

1. Go to the GitHub OAuth application created in the previous section and click on **Edit** to update the settings and `Homepage URL`` to automation portal URL and replace the hostname with the correct hostname in "Redirect URI".




Now we are ready to configure the Execution Environment Builder to push content to the Git repository and pull certified content from the Private Automation Hub in chapter 3.

# Centralized Content Discovery

Centralized Content Discovery provides a unified catalog architecture designed to aggregate automation assets from distributed sources into a single, searchable interface. It eliminates manual discovery efforts by streamlining how platform engineers manage and surface Ansible Content Collections.

Key features include:

- **Multi-Source Aggregation:** Integrates diverse repositories, including GitHub, GitLab, and Private Automation Hub.
- **Continuous Synchronization:** Supports scheduled updates to ensure the catalog remains current with the latest automation assets.
- **Controlled Visibility:** Implements predefined scoping and access policies to manage how users browse and discover available collections.
- **Simplified Traceability:** Enables a "browse-before-build" workflow, allowing developers to identify existing assets and avoid redundant development.

# Unified Catalog Architecture for Multi-Source Aggregation

In modern enterprise automation, content is rarely stored in a single location. Ansible Collections are often dispersed across distributed version control systems (like GitHub or GitLab) and local distribution points (like Private Automation Hub). The **Unified Catalog Architecture** in Ansible Automation Platform 2.7 solves the "siloed content" problem by abstracting the storage layer and providing a single pane of glass for discovery.

## Benefits of Aggregation

- **Reduced Discovery Friction:** Developers can search for a module or collection without needing to know the hosting infrastructure beforehand.
- **Version Visibility:** The catalog tracks metadata across sources, allowing users to compare if an internal, customized collection in the Private Automation Hub is newer or older than an upstream version in GitHub.
- **Policy-Driven Access:** By aggregating content, administrators can apply RBAC to the catalog, ensuring that users only see collections they are authorized to consume, regardless of the source.

## Configuring Supported Sources including GitHub, GitLab, and Private Automation Hub

In the Ansible Automation Platform 2.7 architecture, Centralized Content Discovery acts as a unified catalog, aggregating automation assets from disparate locations into a single, searchable interface. By configuring external sources, administrators ensure that developers and operators have immediate access to validated collections and roles without manual intervention.

## Architectural Overview

The platform allows for the orchestration of multiple content repositories, including:

- **Git Repositories (GitHub/GitLab):** Used for custom automation content, playbooks, and roles managed under version control.
- **Private Automation Hub:** The enterprise source for certified and validated Ansible Content Collections and execution environment definitions.

Administrators maintain granular control over these integrations by defining exactly which repositories are discoverable, applying corporate filtering policies to ensure compliance and security.

## Configuring Content Sources

To ensure the platform has visibility into your automation ecosystem, you must register each source and configure its synchronization parameters.

## Hands-on Lab: Configuring Multi-Source Synchronization

This activity demonstrates how to aggregate content from both a GitHub organization and your enterprise Private Automation Hub.

1. **Navigate to Source Configuration**

   ```
   Log in to the Automation Portal as an administrator and go to the *Collections*
   configuration page.
   ```

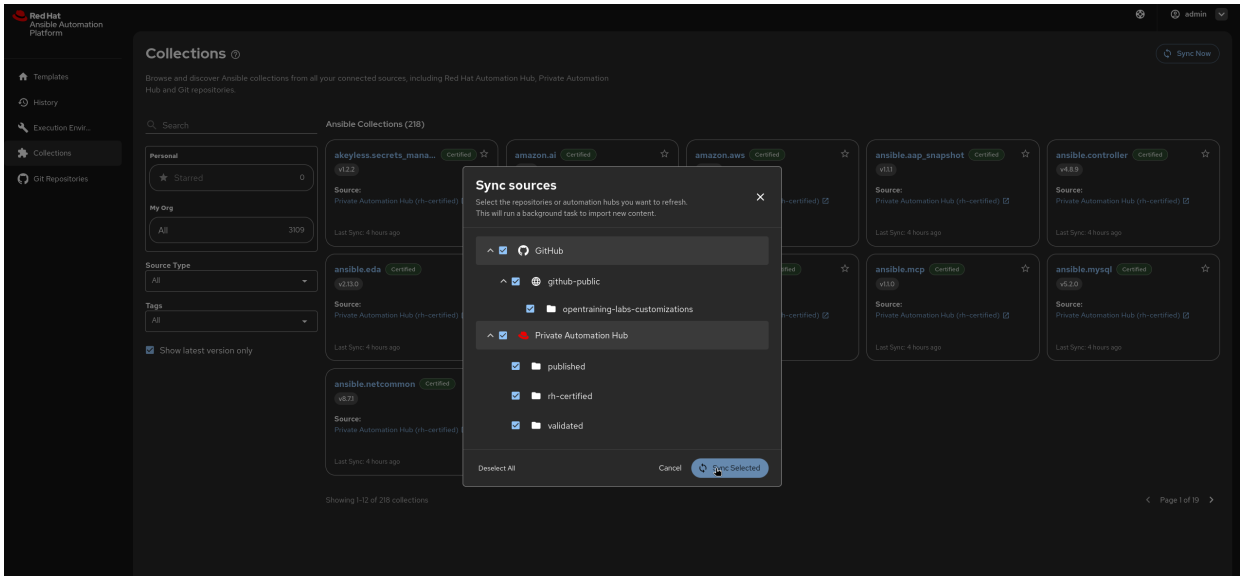
2. **Trigger a Manual Sync** To populate the catalog immediately, perform a manual synchronization:

   Click the **Sync now** button located at the top right of the Sources dashboard.

   In the selection dialog, select both:

   - [x] GitHub Organization: `<your-org>`
   - [x] Private Automation Hub

   Click **Start sync**.




3. **Verify Completion**
   - Monitor the status indicators. The labels will transition from **"Syncing..."** to **"Synced"** or **"Completed"**.
   - Once synced, navigate to the unified catalog to browse the aggregated collections.



Administrators can apply custom filtering to these sources to align with corporate boundaries, ensuring that only approved content enters the developer workflow. Continuous synchronization can be scheduled to ensure the catalog remains updated as new versions of collections are published.

Let 's explore how the Execution Environment (EE) builder can be used to create a custom EE that includes the aggregated content from both GitHub and Private Automation Hub. This allows developers to run automation jobs with the latest validated content without needing to manage dependencies manually.

# Visual Execution Environment Builder

The Visual Execution Environment (EE) Builder is a GUI-based tool that simplifies the creation of portable, reproducible automation runtime images. It replaces complex CLI-based workflows by providing a guided interface to manage the assembly of `ansible-core`, collection dependencies, Python packages, and system libraries.

Key features include:

- **Guided Workflow**: Utilizes a step-by-step wizard to define custom EEs, removing the need to manually author `execution-environment.yml` files.
- **Integrated Discovery**: Enables users to select Ansible Content Collections directly from the centralized discovery catalog.
- **Definition Management**: Automatically generates complete EE definition files with options to download or publish directly to repositories.
- **CI/CD Integration**: Supports automated container image builds through scaffolded CI pipelines and integration with Git-based workflows.
- **Customization**: Allows for the definition of pre-build and post-build steps, alongside the integration of Red Hat domain-specific presets.

# Architecture and Transition from Manual CLI Workflows to GUI-Based Building

The shift from manual command-line interface (CLI) workflows to the Visual Execution Environment (EE) Builder represents a fundamental change in how platform engineers manage automation dependencies. This transition prioritizes repeatability, governance, and user accessibility by abstracting the complexities of container image construction.

## Benefits of the GUI-Based Approach

Transitioning to a GUI-based builder offers several architectural advantages:

- **Standardization:** Instead of disparate scripts, the portal uses a guided wizard that ensures all necessary metadata (Python packages, system libraries, and collection dependencies) is captured in a standardized format.
- **Integrated Dependency Management:** The builder leverages the **Centralized Content Discovery** catalog. Rather than manually typing collection requirements, you browse the synced catalog, ensuring you are using verified, organizationally approved content.
- **Abstraction of Complexity:** Platform engineers no longer need to maintain complex `ansible-builder` logic. The portal handles the underlying container build instructions while presenting a clean, intent-based interface.
- **Automated CI/CD Integration:** The Visual Builder does not just build an image—it scaffolds the entire lifecycle. By publishing definitions to Git and automating pipelines via GitHub Actions, the transition moves from "manual execution" to "automated lifecycle management."

## Architectural Benefits of Reusable Templates

- **Standardization:** Ensures all teams build EEs using approved base images and vetted Ansible Collections.
- **Reduced Complexity:** New users can start with a pre-configured template rather than defining complex dependencies from scratch.
- **Version Control Integration:** Templates stored in Git track changes to your automation requirements over time.
- **Self-Service Enablement:** Enables a "browse-before-build" workflow where developers can discover and deploy existing, compliant EE configurations.



By centralizing EE builds in the portal, you regain control over the "What" and "How" of your automation runtime. Every GUI-based build is tracked and gated by admin-defined access policies, effectively turning your EE creation process into a governed, transparent activity.

## Integrating Red Hat Domain-Specific Presets and Base Container Images

In the Automation portal Visual Execution Environment (EE) Builder, selecting the right foundation is an important step for achieving security, compliance, and performance. Red Hat provides domain-specific presets as curated starting points, helping architects avoid building an EE from scratch.

### Understanding Domain-Specific Presets

These presets provide pre-configured Ansible Collections, Python dependencies, and system libraries tailored to specific operational domains. By starting with a validated foundation, we can reduce configuration effort and build EEs more consistently for the required use case.

- **Networking:** Includes `ansible.netcommon`, `cisco.ios`, and `arista.eos` collections, alongside system-level utilities for network device communication.
- **Security:** Focused on compliance auditing and vulnerability management, featuring `ansible.posix` and various security-hardened utility packages.
- **Cloud:** Pre-loaded with cloud-provider-specific SDKs (AWS, Azure, GCP) and the corresponding `community.cloud` or `amazon.aws` collections.

Using these presets ensures that your EEs adhere to organizational standards for library versions and security patching, reducing the maintenance overhead of managing disparate `execution-environment.yml` files.



When operating in disconnected environments, ensure your local registry is mirrored correctly using `skopeo copy`. As noted in the platform documentation, the use of SHA256 digests is recommended to ensure that your custom EE builds remain immutable and consistent across development and production OCP clusters.

# Utilizing the Step-by-Step Wizard for Custom EE Creation

Creating Execution Environments (EEs) traditionally required manual management of `execution-environment.yml` files, complex dependency resolution, and deep knowledge of container build contexts. The Visual Execution Environment Builder within the Automation Portal abstracts this complexity, providing a guided, wizard-driven interface to ensure standardized, error-free images.

## Overview of the Wizard Workflow

The wizard workflow is designed to move from high-level intent to a concrete container definition. By leveraging Red Hat-supported presets, the wizard minimizes the "guesswork" involved in selecting base images and dependency formats.

The process follows a logical progression: 1. **Definition Naming and Scoping:** Defining the EE purpose and metadata. 2. **Base Image Selection:** Choosing from Red Hat-optimized, pre-validated base images (e.g., `ee-minimal-rhel9`, `ee-supported-rhel9`). 3. **Dependency Injection:** Declarative selection of Ansible Content Collections, Python packages (`requirements.txt`), and system-level libraries (`bindep.txt`). 4. **Build Customization:** Defining pre-build and post-build shell commands to handle non-standard configuration tasks. 5. **Review and Validation:** Real-time generation of the underlying YAML definition.

## Integrating Presets and Dependencies

The wizard leverages Red Hat domain-specific presets to ensure security and compliance. When you select a base image, the wizard automatically pulls the relevant metadata, preventing common version mismatches between the OS layer and the Ansible Automation Platform engine.

- **Ansible Content Collections:** Users can search and select collections directly from the catalog. The wizard manages the `requirements.yml` structure, ensuring all dependencies are pinned correctly.
- **System Libraries:** Rather than manually crafting complex `Dockerfile` layers, users add system packages via a simple list interface, which the wizard translates into an optimized `bindep.txt` configuration.



The EE definition workflow generates a reusable Backstage software template (e.g., `<ee-name>-template.yaml`). This allows organizations to move from one-off EE creation to a "Golden Image" catalog, where team members can provision standardized environments based on pre-approved templates.

## Execution Environment CI/CD Architecture

The main idea is to treat the Execution Environment (EE) definition as code. Instead of managing EEs manually through the portal, we store the definition and its dependencies in Git. This provides **version control, consistency, auditability, and reproducibility**, while allowing the EE build and publishing process to be automated through a **CI/CD pipeline**.

Overall, this creates an automated flow from **Git change** → **CI/CD pipeline** → `ansible-builder` → **container image** → **registry** → **Ansible Automation Platform**, reducing manual effort while improving consistency and traceability.

### Best Practices for Wizard Usage

- **Use Version Pinning:** Always specify versions for collections and Python packages within the wizard to ensure reproducible builds.
- **Leverage Post-Build Steps:** Use the post-build section for cleanup tasks or custom environment variable injection rather than manual Dockerfile modifications.
- **Catalog Early:** Once your EE definition is stable, export the generated Backstage template to your private catalog to enable self-service consumption by other developers.

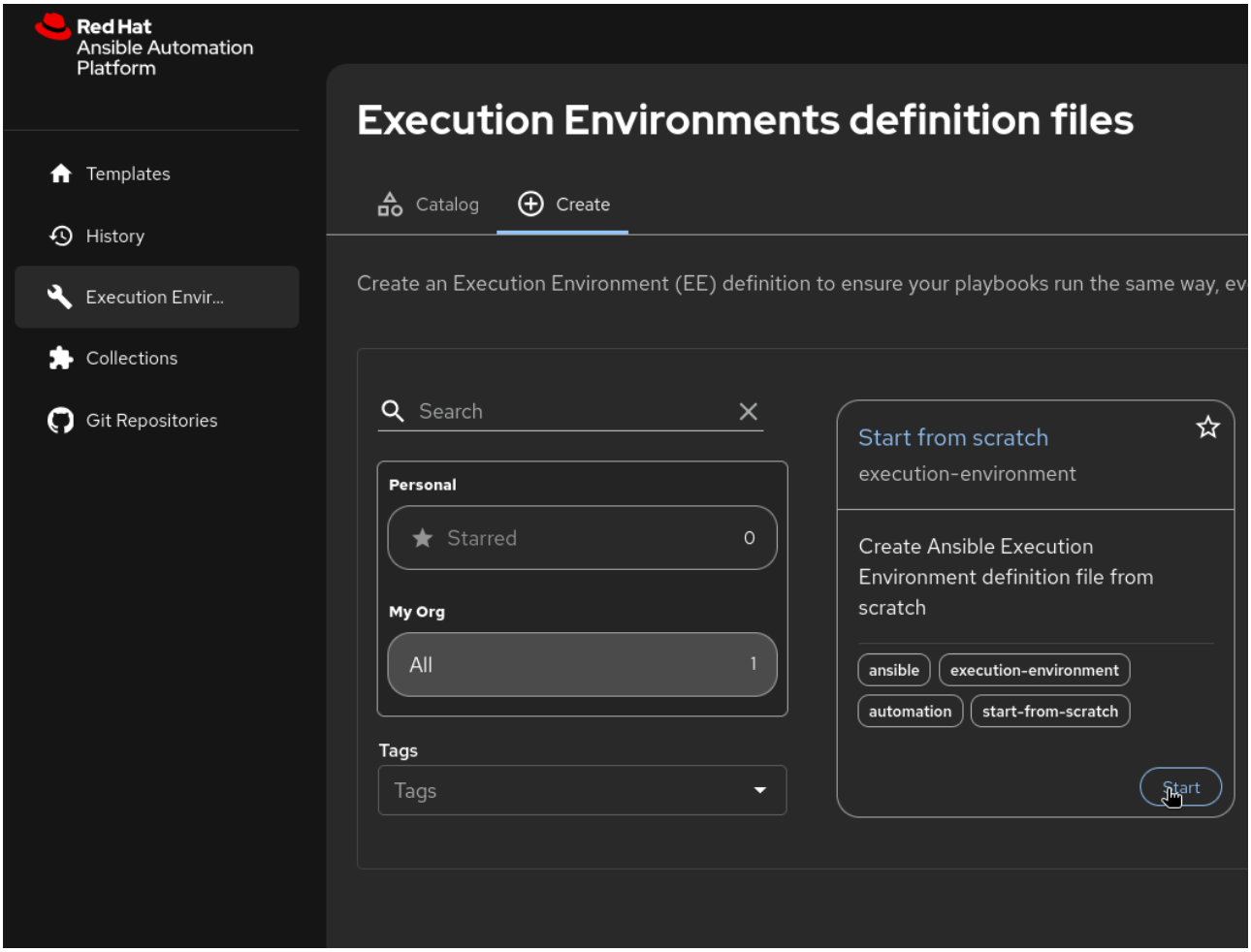
## Hands-on Lab: Content Discovery and Visual EE Building

In this lab, you will leverage the Automation Portal to unify disparate automation sources and transition from manual image creation to a streamlined, GUI-based Execution Environment (EE) workflow.

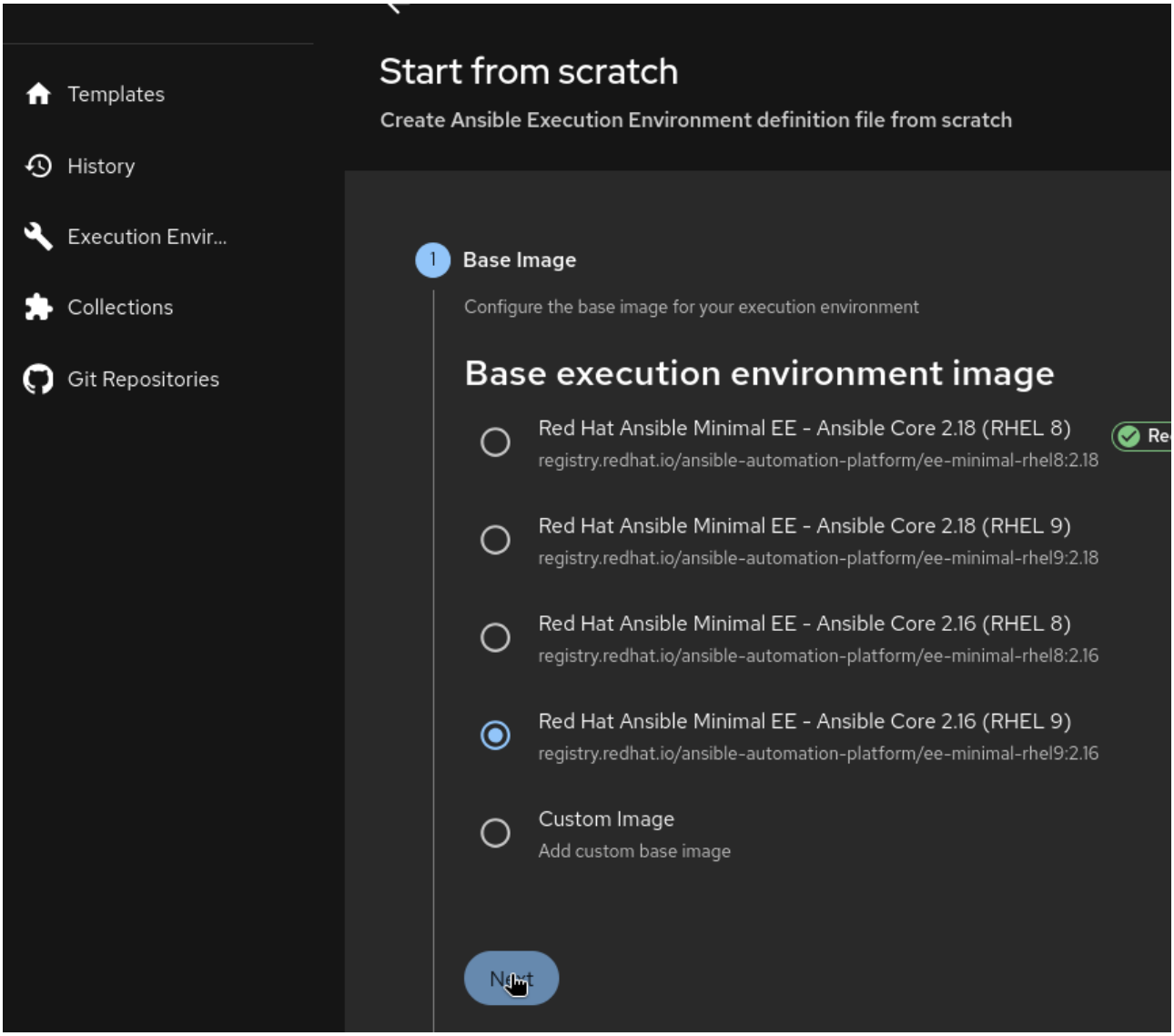
### Building a Custom Execution Environment

We will now use the Visual EE Builder to generate a container definition without writing a single line of YAML or `ansible-builder` configuration files.

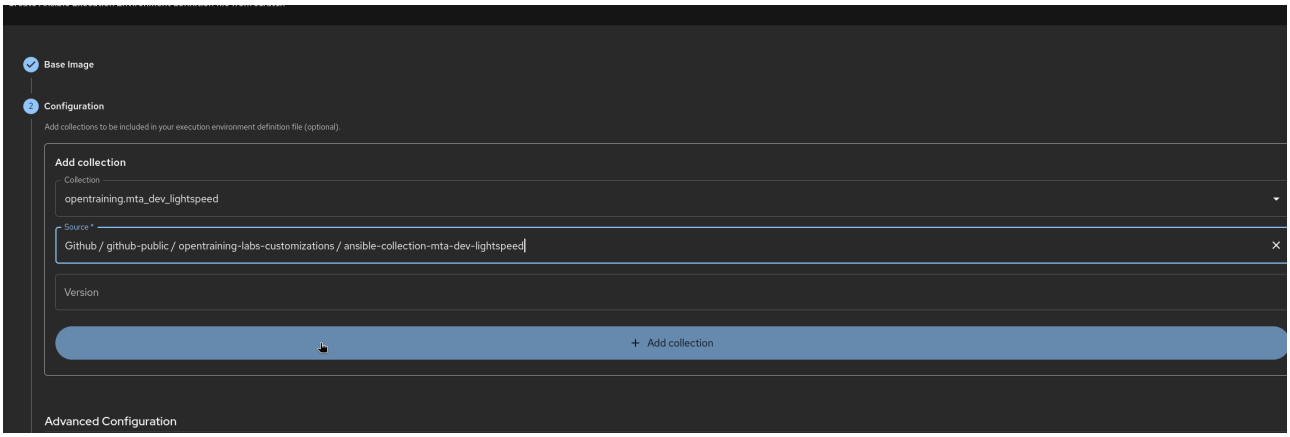
1. Login to the Automation Portal and navigate to **Execution Environments** > Created > **Start from scratch** Template. This will launch the Visual EE Builder wizard.



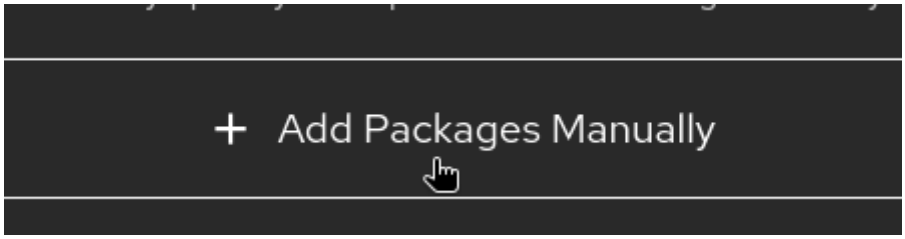

2. Select the Execution Environment images which you wanted to use as base image. I will be using the **Red Hat Ansible Minimal EE - Ansible Core 2.16 (RHEL 9)** as base image for this lab:

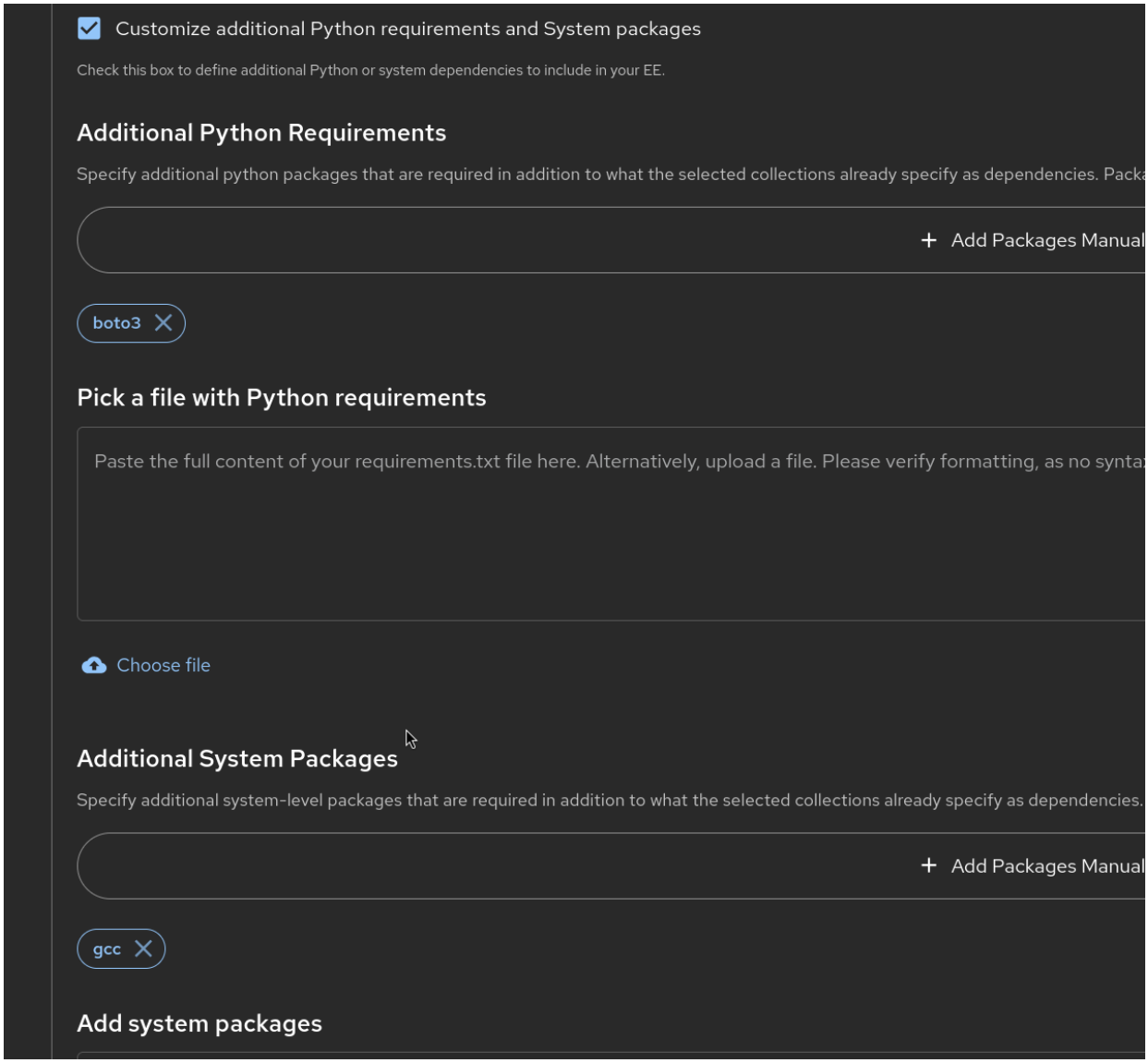



3. Then we can add collection you can skip and move to the next step. if you don't have any collection to add or use community collection to add. I am adding a custom collection **opentraining.mta_dev_lightspeed** from my repo.




4. **Inject Dependencies:**
   - Click the **Dependencies** tab.
   - In the **Python Requirements** section, add `boto3`.
   - In the **System Packages** section, add `gcc`.



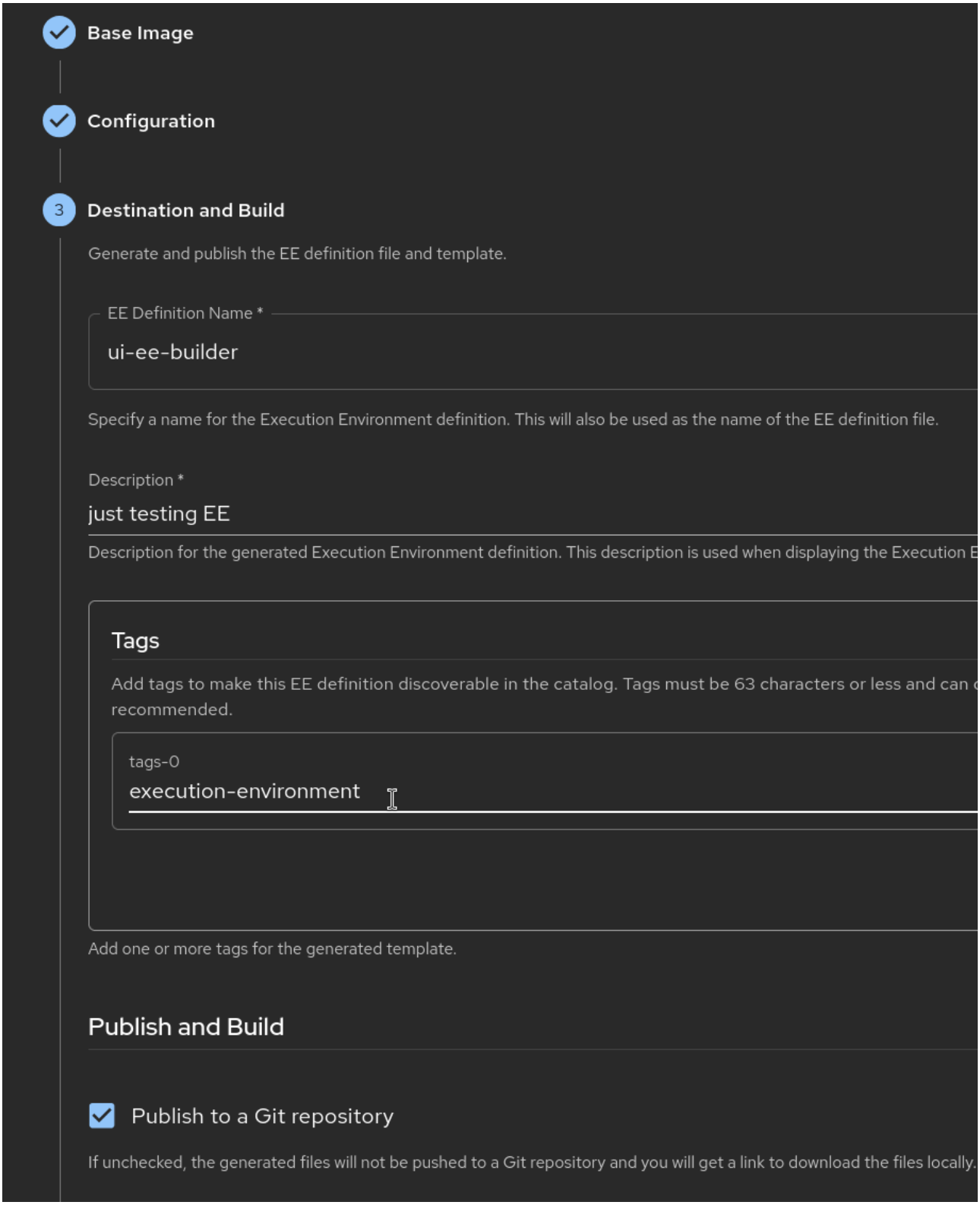

5. **Define Build Steps:**
   - Navigate to **Custom Steps**.
   - Add a **Pre-build** step: Enter a shell command to verify the file system structure.
   - Add a **Post-build** step: Include a command to run a security scan script (e.g., `ansible-lint -v`).



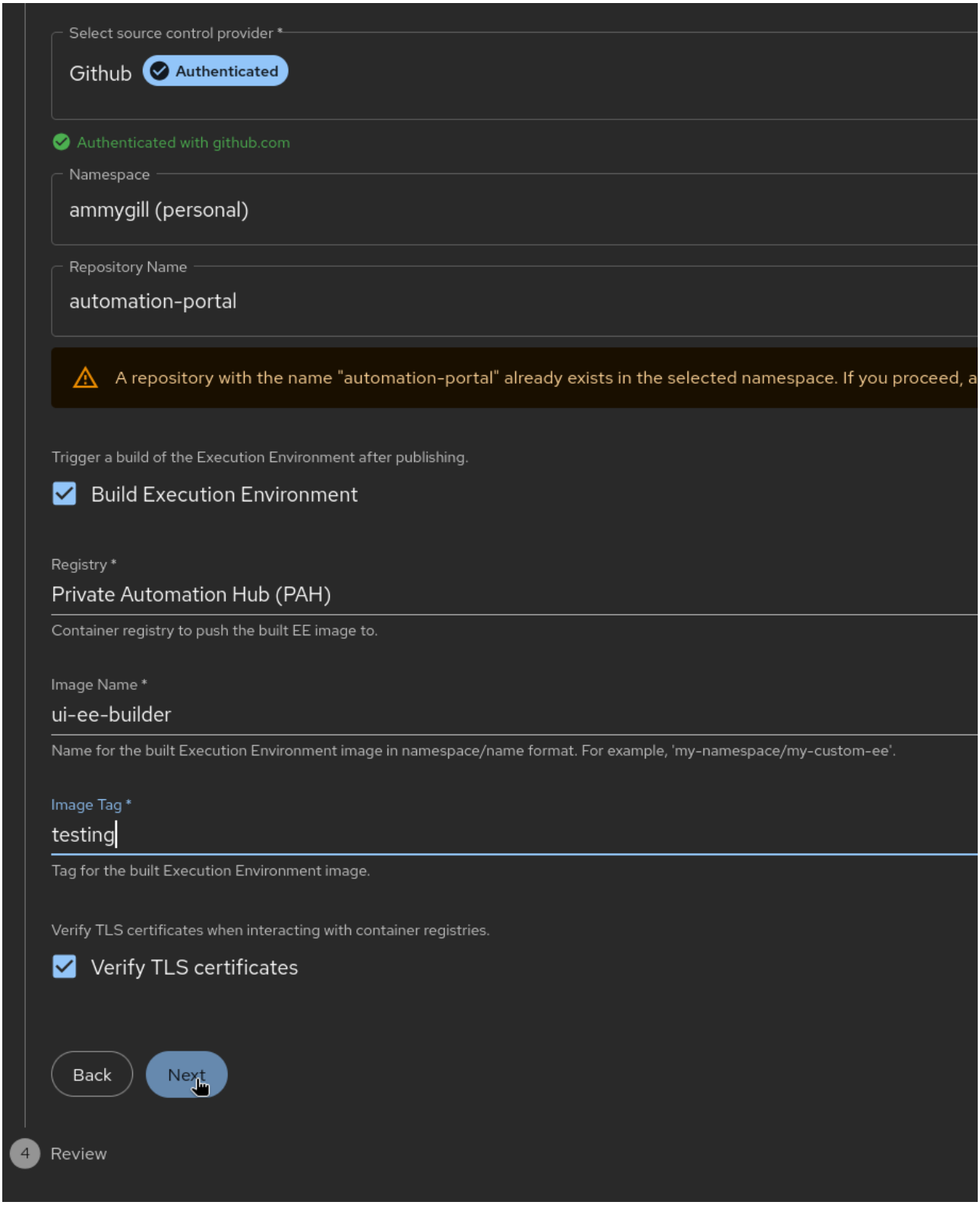
I am keeping the pre-build and post-build steps empty for this lab. You can add your own pre-build and post-build steps as per your requirement.

1. **Destination and Build** Step here you just have to provide the information about the EE names tag with which you wanted to create and with what names you wanted to publish this EE imaged to the registry. Please check the below screenshot on what information i am providing

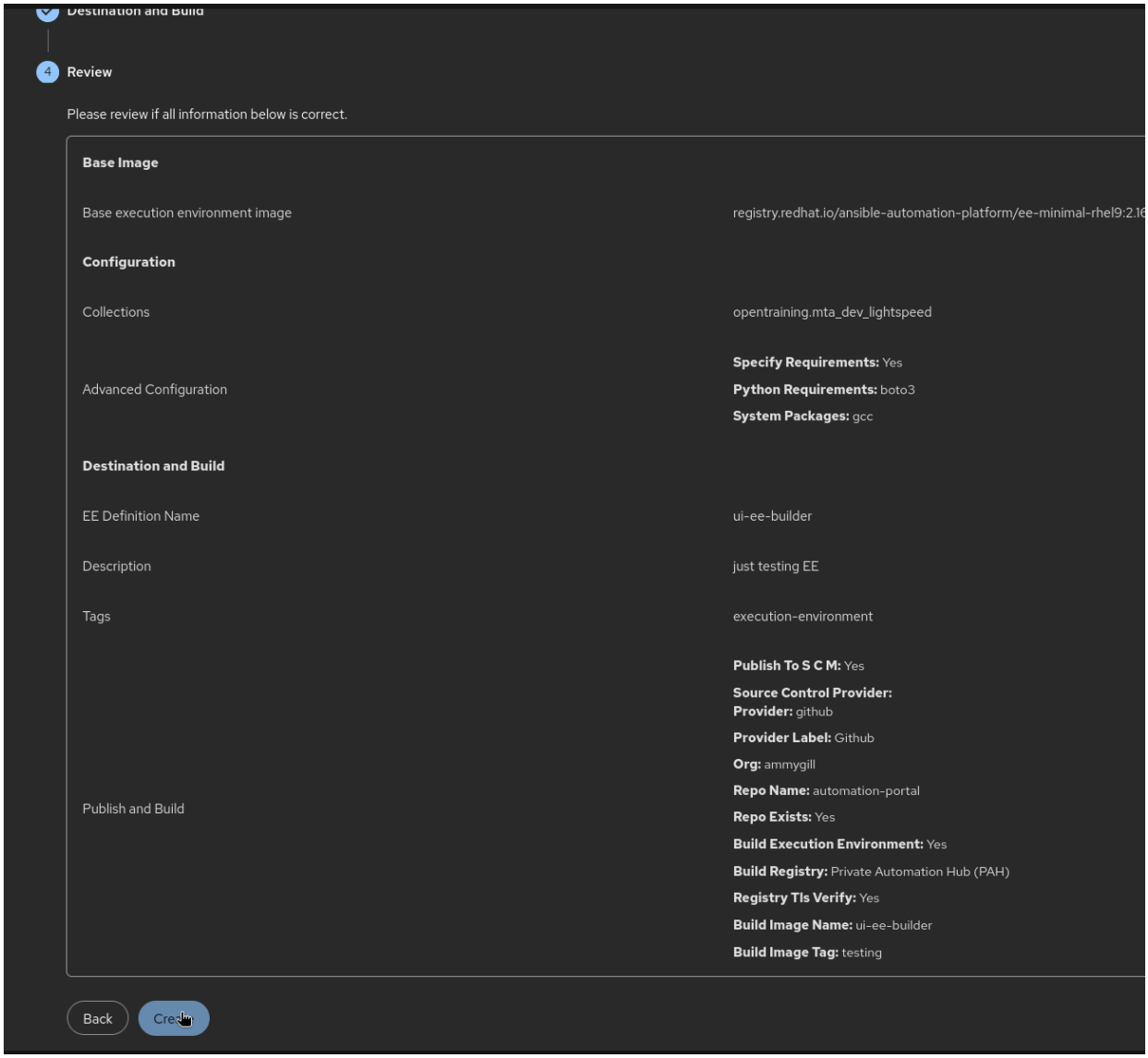
to create EE.



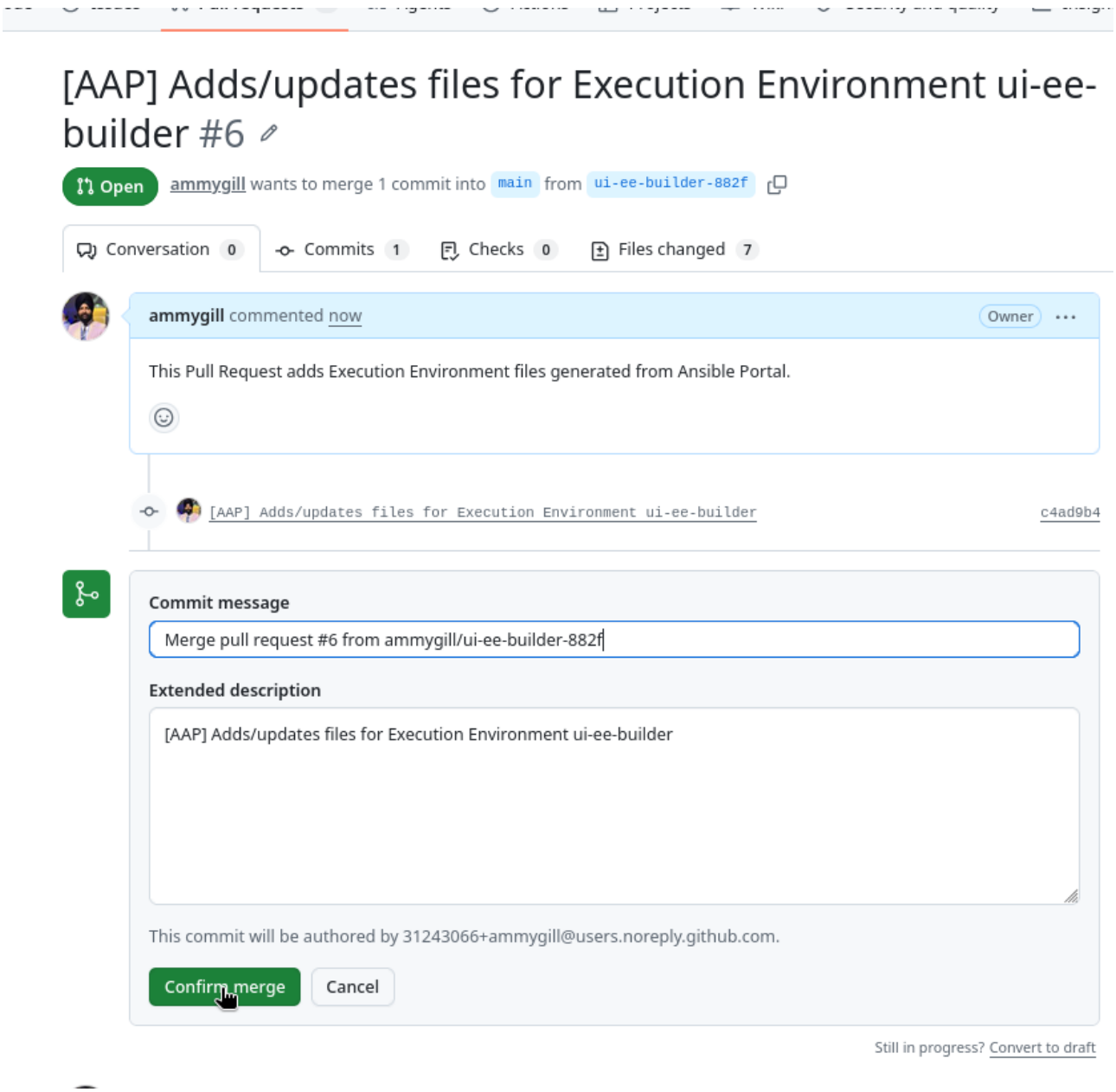

2. Lets Work on the step **Publish and Build** which will be used to publish the EE definition to the GitHub repository and trigger the CI/CD pipeline to build the container image and push it to the registry.

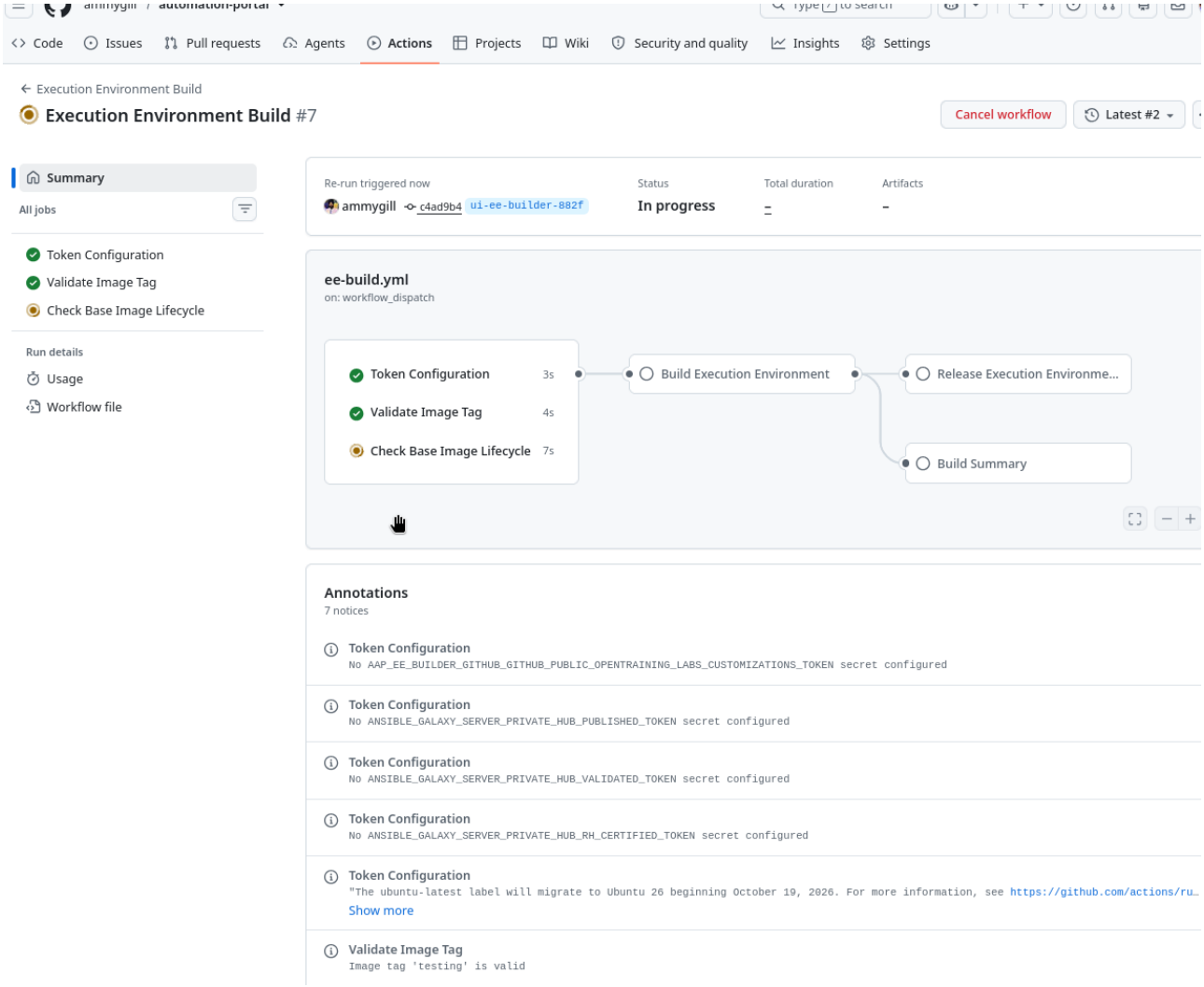



3. **Finalize and Publish:**
    - Review the generated preview.
    - Click **Save & Publish**. Select your GitHub repository; the builder will automatically push the `execution-environment.yml` and associated metadata to your repository.

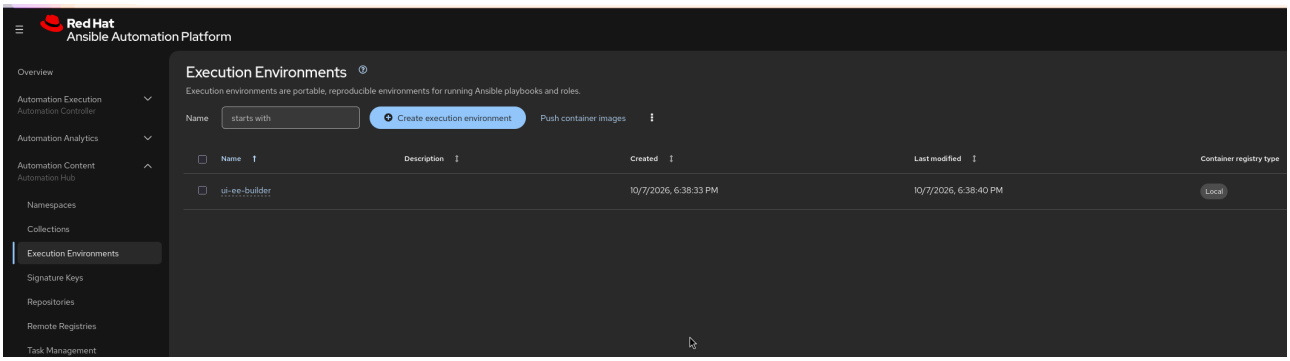



4. The PR will be raised to the GitHub repository and you can see the PR in your GitHub repository. You can merge the PR to the main branch and this will trigger the CI/CD pipeline to build the container image and push it to the registry as per credentials saved in the Chapter 1.


[AAP] Adds/updates files for Execution Environment ui-ee-builder #6
Open
ammygill wants to merge 1 commit into main from ui-ee-builder-882f
Conversation 0
Commits 1
Checks 0
Files changed 7
ammygill commented now
Owner
This Pull Request adds Execution Environment files generated from Ansible Portal.
[AAP] Adds/updates files for Execution Environment ui-ee-builder
c4ad9b4
Commit message
Merge pull request #6 from ammygill/ui-ee-builder-882f
Extended description
[AAP] Adds/updates files for Execution Environment ui-ee-builder
This commit will be authored by 31243066+ammygill@users.noreply.github.com.
Confirm merge
Cancel
Still in progress? Convert to draft




5. Once the Builde is completed you can see the image is pushed to the registry in this case it will be on the Automation content in the ansible automation platform and you can see the image in the registry.




Congratulations! You have successfully created a custom Execution Environment using the Visual EE Builder, integrated it with GitHub for version control, and automated the build and deployment process through CI/CD pipelines. This approach ensures that your EEs are consistent, reproducible, and compliant with organizational standards.

# Troubleshooting Tips

- **Sync Failures:** If sources fail to sync, check the **Activity Log** in the Portal to verify API token permissions and network connectivity to the remote source.

- **Dependency Conflicts:** If a collection requires a specific Python version not supported by your base image, the builder will provide a validation error. Switch to a more compatible base image (e.g., `ee-supported-rhel9`).
- **RBAC Restrictions:** If you cannot see the Visual Builder, ensure your user account is assigned the `Automation Administrator` or `EE Builder` role.

## Verification

1. Verify that the build process has been triggered by navigating to your linked GitHub repository. You should see a `build/` directory populated with your definition, confirming that the Visual Builder has successfully handed off the configuration to your CI/CD pipeline.
2. You can always re run the workflow in the **Action** section of the github repository to trigger the build and push the image to the registry. You can also check the logs of the workflow to see if there is any error in the build process.

# Appendix

1. Red Hat Documentation for Ansible Automation Platform. You can find the official documentation at: https://docs.redhat.com/en/documentation/red_hat_ansible_automation_platform/2.7